College of Engineering and Technology

Department of computer science

Java Programming Lab Manual

Dear Students

This manual covers all required steps and practical exercises on Java Programming. All steps are necessary to accomplish one given task. Please perform without any exhaustion. Dear our students, we need your suggestions and comments as well as questions and problems which could be included in this lab module. In general, speaking it is open for advanced.

Be free to Contact as by the following Address

1. Addisu G. (BSc)
   Email: addisug7th@gmail.com
2. Hulunayehu M. (MSc)
   Email: hulumolla1@gmail.com

Table of Contents

Chapter One 1
1. Introduction to Java Programming 1
1.1. Overview of java programing 1
1.2. Variable Declaration 2
1.3. Arithmetic and Logical Operations 2
1.4. Java Conditional and Loop Statements 3
Chapter Two 8
2. Java Applets 8
Chapter Three 13
3. Java GUI with Swing 13
3.1. Introduction to Java Swing 13
3.2. Basic Components and Layouts 13
3.3. Event Handling 15
3.4. Advanced GUI Components 17
3.5. JavaFX Layout Components 24
3.6. Basic UI Controls 31
3.7. Event Handlers in Java Programming 37
3.8. UI Controls in Java Programming 43
3.9. Working with Shapes 50
3.10. CSS Styling in Java Programming (JavaFX) 55
3.11. Property Bindings in Java Programming (JavaFX) 60
3.12. Graphics and Animation in (JavaFX) 66
Chapter Four 75
4. File Input-Output 75
4.5. Reading and Writing Byte Data using FileInputStream and FileOutputStream 78
4.6. Handling Exceptions in File I/O 79
4.7. BufferedWriter and BufferedReader 81
4.8. DataInputStream and DataOutputStream 88
4.9. Multi-threading Concepts 93
4.10. Thread Synchronization 95
Chapter Five 101
5. Socket Programming 101
5.1. Introduction to Socket and ServerSocket Classes 101

Chapter six 108
6. Remote Method Invocation (RMI) 108
6.1. Introduction to RMI 108
6.2. Architecture of RMI 108
6.3. Steps to Implement RMI 108
Chapter Seven 114
7. Java Database Connectivity (JDBC) 115
7.1. Introduction 115
7.2. Setting up the Database 115
7.3. Connecting Java to a Database 116
7.4. Creating and Executing SQL Statements 116
7.5. Querying and Manipulating Results of SQL Statements 122
Chapter Eight 129
8. Servlet Programming 129
8.1. Servlet Lifecycle 129
8.2. Compiling and Deploying a Servlet 130
8.3. Accessing Data from HTML Form 131
8.4. Client Request and Server Response 132
8.5. Understanding HTTP Codes 136
References: 138

Course Objectives

At the end of this course, students should be able to:

- Carry out design and development of complex elements, such as user interfaces, multiprocessing,and fault-tolerant components;
- Write TCP/IP Client Server applications using Sockets;
- Write Java applications using the JDBC to make database independent queries; and Call methods remotely.

Chapter One

1. Introduction to Java Programming

Objective:

- To understand the basics of Java programming.
- To declare and initialize variables.
- To perform arithmetic and logical operations in Java.

Topics Covered:

- Overview of Java Programming
- Variable Declaration
- Arithmetic and Logical Operations

1.1. Overview of java programing

Java is a high-level, object-oriented programming language widely used for its portability, security and performance. This session introduces the syntax and structure of Java programs.

Key Concepts:

- **Java Development Kit (JDK):** Includes tools for compiling and running Java programs.
- **Java Runtime Environment (JRE):** Needed to run Java applications.
- **Java Virtual Machine (JVM):** Allows Java to run on any platform by providing a runtime environment.

Exercise 1: Write and execute a basic "Hello World!" Java program.

```
public class HelloWorld {
public static void main(String[] args) {
System.out.println("Hello, World!");
}
}
```

1. Create a Java file named HelloWorld.java.
2. Compile it using javac HelloWorld.java.
3. Run the program with java HelloWorld.

1.2. Variable Declaration

Variables are containers that hold data values. In Java, each variable must be declared with a data type before use.

Key Data Types:

- **int:** for integers.
- **double:** for decimal numbers.
- **boolean:** for true/false values.
- **char:** for single characters.
- **String:** for sequences of characters.

Syntax for Declaration and Initialization:

```
int number = 10;
double price = 25.99;
boolean isAvailable = true;
char grade = 'A';
String name = "Java";
```

Exercise 2: Declare variables for a student's profile, including name, age, grade, and isEnrolled.

1.3. Arithmetic and Logical Operations

Arithmetic Operations

Java supports the following arithmetic operations:

Operation	Symbol	Example	Result
Addition	+	a + b	Sum of a and b
Subtraction	-	a – b	Difference of a and b
Multiplication	*	a * b	Product of a and b
Division	/	a / b	Quotient of a and b
Modulus	%	a % b	Remainder of a / b

Example:

```
int a = 10;
int b = 5;
System.out.println("Sum: " + (a + b));
```

```
System.out.println("Difference: " + (a - b));
System.out.println("Product: " + (a * b));
System.out.println("Quotient: " + (a / b));
System.out.println("Remainder: " + (a % b));
```

Logical Operations

Logical operators in Java are used to combine conditional statements.

Operation	Symbol	Example	Result
Logical AND	&&	(a > 5) && (b < 10)	true if both true
Logical OR	||	(a > 5) || (b < 10)	True if one is true
Logical NOT	!	!(a > 5)	true if a > 5 is false

Example:

```
boolean result1 = (10 > 5) && (5 < 10); // true
boolean result2 = (10 < 5) || (5 < 10); // true
boolean result3 = !(10 > 5); // false
```

Exercise 3: Write a program that:

1. Declares two integer variables x and y.
2. Performs arithmetic operations and displays the results.
3. Uses logical operators to compare the values of x and y and displays the results.

Summary:

- This lab introduces basic Java syntax, variable declaration, and operations.
- Practice these concepts with simple Java programs to strengthen your understanding.

1.4. Java Conditional and Loop Statements

Objective:

- To understand and use conditional statements in Java.
- To use loop structures for repetitive tasks.
- To apply logic to control program flow based on conditions and loops.

Topics Covered:

- Java Conditional Statements
- Java Loop Statements

1.4.1. Java Conditinal Statements

Conditional statements allow a program to make decisions based on certain conditions. In Java, the main conditional statements are **if**, else if, else, and switch.

If, else if, and else Statements

Syntax

```
if (condition) {
    // Code to execute if condition is true
} else if (anotherCondition) {
   // Code to execute if anotherCondition is true
} else {
    // Code to execute if all conditions are false
}
```

Example:

```
int number = 10;
if (number > 0) {
   System.out.println("The number is positive.");
} else if (number < 0) {
   System.out.println("The number is negative.");
} else {
   System.out.println("The number is zero.");
}
```

Exercise 1: Write a Java program that

1. Takes an integer input from the user.
2. Checks if the integer is even or odd, and displays the result.

Switch Statement

- The switch statement provides a way to handle multiple conditions based on an expression's value.

Syntax:

```
switch (expression) {
   case value1:
      // Code to execute if expression equals value1
      break;
   case value2:
      // Code to execute if expression equals value2
      break;
   default:
      // Code to execute if none of the cases match
}
```

Example:

```
int day = 3;
switch (day) {
   case 1:
 System.out.println("Monday");
      break;
   case 2:
System.out.println("Tuesday");
      break;
   case 3:
System.out.println("Wednesday");
      break;
   default:
      System.out.println("Invalid
day");
}
```

Exercise 2: Write a program that

1. Accepts a number representing a month (1 for January, 2 for February, etc.).
2. Uses a switch statement to print the name of the month.

1.4.2. Java Loop Statement

Loops in Java allow you to execute a block of code repeatedly as long as a specified condition is met. Java supports three primary loop types: for, while, and do-while.

For Loop

Syntax

```
for (initialization; condition; update)
```

```
{
    // Code to be executed
}
```

Example:

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Iteration: " + i);
}
```

Exercise 3: Write a program that

1. Uses a for loop to display the numbers from 1 to 10.

While Loop

- The while loop executes as long as the condition is true.

Syntax:

```
while (condition) {
    // Code to be executed
}
```

Example:

```
int i = 1;
while (i <= 5) {
    System.out.println("Count: " + i);
    i++;
}
```

Exercise 4: Write a Java program that:

1. Uses a while loop to print the even numbers between 1 and 20.

do-while Loop

- The do-while loop executes at least once, even if the condition is false.

Syntax:

```
do {
    // Code to be executed
} while (condition);
```

Example:

```
int i = 1;
do {
```

```
        System.out.println("Number: " + i);
        i++;
    } while (i <= 5);
```

Exercise 5: Write a Java program that:

1. Uses a do-while loop to display numbers from 1 to 5.

Advanced Exercises

1. **FizzBuzz Problem:** Write a Java program that:
   - Uses a loop to iterate numbers from 1 to 50.
   - Prints "Fizz" for numbers divisible by 3, "Buzz" for numbers divisible by 5, and "FizzBuzz" for numbers divisible by both 3 and 5.
2. **Prime Numbers:** Write a program that:
   - Uses **for loop** to find and print all prime numbers between 1 and 100.
3. **Factorial Calculation:** Write a Java program that:
   - Uses a while or for loop to calculate the factorial of a given integer.

Summary:

- This lab introduces conditional statements (if, else if, else, switch) and loops (for, while, do-while).
- Practice these structures to control the flow of your programs based on conditions and repetitive tasks.

Chapter Two

2. Java Applets

Objective:

- To understand the basics of Java Applets.
- To create and run a simple Java Applet.
- To learn basic methods and event handling in Applets.

Topics Covered:

1. Introduction to Java Applets
2. Basic Structure of a Java Applet
3. Applet Lifecycle Methods
4. Graphics and Drawing in Applets
5. Event Handling in Applets

2.1. Introduction to Java Applets

Java Applets are small Java applications that run within a web browser or an applet viewer. Applets are part of Java's java.applet package and java.awt package for graphics and UI components.

Key Components:

Applet Class: The base class for applets.

Applet Viewer: A tool to test and view applets (usually by running appletviewer command in Java).

Note: Java Applets require the java.applet.Applet package and need a .html file to run in an applet viewer.

2.2. Basic Structure of a Java Applet

To create an applet, extend the Applet class and implement its lifecycle methods.

Example: Simple Applet Program

```
import java.applet.Applet;
import java.awt.Graphics;
public class HelloWorldApplet extends Applet {
    public void paint(Graphics g) {
```

```
        g.drawString("Hello, World!", 20, 20);
    }
}
```

Steps:

1. Save the program as HelloWorldApplet.java.
2. Compile it using javac HelloWorldApplet.java.
3. Create an HTML file to run the applet.

HTML file (HelloWorldApplet.html):

```
<applet code="HelloWorldApplet.class" width="300" height="100"></applet>
```

4. Run the applet using the applet viewer:
   appletviewer HelloWorldApplet.html

2.3. Applet Lifecycle Methods

The main lifecycle methods of an applet are:

- init(): Initializes the applet; runs once.
- start(): Starts or resumes the applet; runs each time the applet is (re)started.
- stop(): Pauses the applet; runs when the applet is stopped.
- destroy(): Cleans up resources before the applet is destroyed.

Example: Using Lifecycle Methods

```
import java.applet.Applet;
import java.awt.Graphics;
public class LifecycleApplet extends Applet {
    public void init() {
        System.out.println("Applet initialized.");
    }
    public void start() {
        System.out.println("Applet started.");
    }
    public void stop() {
        System.out.println("Applet stopped.");
```

```
    }
    public void destroy() {
        System.out.println("Applet destroyed.");
    }
    public void paint(Graphics g) {
        g.drawString("Applet Lifecycle Example", 20, 20);
    }
}
```

Exercise 1: Modify the LifecycleApplet to display messages on the applet screen (using g.drawString) for each lifecycle method.

2.4. Graphics and Drawing in Applets

Applets use the paint(Graphics g) method to draw shapes, text, and images.

Common Drawing Methods:

- g.drawString(String text, int x, int y): Draws a string.
- g.drawRect(int x, int y, int width, int height): Draws a rectangle.
- g.drawOval(int x, int y, int width, int height): Draws an oval.
- g.drawLine(int x1, int y1, int x2, int y2): Draws a line.

Example: Drawing Shapes

```
import java.applet.Applet;
import java.awt.Graphics;
public class ShapeApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Drawing Shapes", 20, 20);
        g.drawRect(20, 40, 50, 30);
        g.drawOval(100, 40, 50, 50);
        g.drawLine(20, 100, 150, 100);
    }
}
```

Exercise 2: Write a Java Applet that:

1. Draws a house shape using lines, rectangles, and triangles.
2. Displays the name "My House" below the house drawing.

4.5. Event Handling in Applets

Event handling allows an applet to respond to user actions, such as mouse clicks and keyboard input. To handle events in an applet, implement the java.awt.event package interfaces like **MouseListener** or **ActionListener.**

Example: Mouse Event Handling

```
import java.applet.Applet;
import java.awt.Graphics;
import java.awt.event.MouseListener;
import java.awt.event.MouseEvent;
public class MouseApplet extends Applet implements MouseListener {
    String message = "";
    public void init() {
        addMouseListener(this);
    }
    public void paint(Graphics g) {
        g.drawString(message, 20, 20);
    }
    public void mouseClicked(MouseEvent e) {
        message = "Mouse clicked at (" + e.getX() + ", " + e.getY() + ")";
        repaint();
    }
    public void mouseEntered(MouseEvent e) {}
    public void mouseExited(MouseEvent e) {}
    public void mousePressed(MouseEvent e) {}
    public void mouseReleased(MouseEvent e) {}
}
```

Exercise 3: Modify the MouseApplet program to:

1. Display "Mouse Entered" when the mouse pointer enters the applet area.
2. Display "Mouse Exited" when it leaves the area.

Summary:

- This lab introduces Java Applets, focusing on their lifecycle, graphics, and event handling.
- Practice these concepts to create interactive graphical applications.

Chapter Three

3. Java GUI with Swing

Objective:

- To learn how to create simple GUI applications in Java using Swing.
- To understand the basics of GUI components and event handling.
- To design interactive applications with various GUI elements like buttons, labels, text fields, and more.

Topics Covered:

1. Introduction to Java Swing
2. Basic Components and Layouts
3. Event Handling
4. Building a Simple GUI Application
5. Advanced GUI Components

3.1. Introduction to Java Swing

Java Swing is part of the Java Foundation Classes (JFC) that provides a rich set of lightweight GUI components.

- **JFrame:** The main window for your application.
- **JPanel:** A container that holds and organizes components.
- **JLabel:** Used to display text or images.
- **JButton:** A clickable button.
- **JTextField:** A field for text input.

3.2. Basic Components and Layouts

Creating a JFrame

The JFrame **serves as the window for your GUI application.**

Example: Creating a Basic JFrame

```
import javax.swing.JFrame;
public class MyFrame extends JFrame {
    public MyFrame() {
        setTitle("My First GUI");
        setSize(400, 300);
```

```
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
    public static void main(String[] args) {
        new MyFrame();
    }
}
```

3.2.1 Adding Components to JFrame

Swing components can be added to the frame. To position components, Java uses layout managers like FlowLayout, BorderLayout, GridLayout, etc.

Example: Adding JLabel and JButton

```
import javax.swing.*;
public class MyFrame extends JFrame {
    public MyFrame() {
        setTitle("Simple GUI");
        setSize(400, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Components
        JLabel label = new JLabel("Enter your name:");
        JTextField textField = new JTextField(15);
        JButton button = new JButton("Submit");
        // Add components to the frame
        setLayout(new FlowLayout());
        add(label);
        add(textField);
        add(button);
        setVisible(true);
    }
    public static void main(String[] args) {
        new MyFrame();
    }
```

```
}
```

Exercise 1: Modify the above example to:

1. Add another label that displays the message "Hello, [Name]!" when the button is clicked.

3.3. Event Handling

Event handling allows us to respond to user actions, such as button clicks or text input.

3.3.1 ActionListener Interface

To handle button clicks, use the **ActionListener** interface and its **actionPerformed** method.

Example: Button Click Event Handling

```
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class MyFrame extends JFrame implements ActionListener {
   JTextField textField;
   JLabel label;
   JButton button;
     public MyFrame() {
       setTitle("Event Handling Example");
       setSize(400, 200);
       setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
       label = new JLabel("Enter your name:");
       textField = new JTextField(15);
       button = new JButton("Submit");
       button.addActionListener(this);
       setLayout(new FlowLayout());
       add(label);
       add(textField);
       add(button);
       setVisible(true);
   }
   public void actionPerformed(ActionEvent e) {
```

```
        String name = textField.getText();
        label.setText("Hello, " + name + "!");
    }
    public static void main(String[] args) {
        new MyFrame();
    }
}
```

Exercise 2: Modify this program to clear the text field after displaying the greeting message.

3.3.2. Building a Simple GUI Application

Let's create a simple calculator GUI that adds two numbers entered by the user.

Example: Simple Calculator

```
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class Calculator extends JFrame implements ActionListener {
    JTextField num1, num2, result;
    JButton addButton;
    public Calculator() {
        setTitle("Simple Calculator");
        setSize(300, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JLabel label1 = new JLabel("Number 1:");
        num1 = new JTextField(10);
        JLabel label2 = new JLabel("Number 2:");
        num2 = new JTextField(10);
        JLabel labelResult = new JLabel("Result:");
        result = new JTextField(10);
        result.setEditable(false);
        addButton = new JButton("Add");
        addButton.addActionListener(this);
```

```
        setLayout(new FlowLayout());
        add(label1);
        add(num1);
        add(label2);
        add(num2);
        add(addButton);
        add(labelResult);
        add(result);
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e) {
        int number1 = Integer.parseInt(num1.getText());
        int number2 = Integer.parseInt(num2.getText());
        int sum = number1 + number2;
        result.setText(String.valueOf(sum));
    }
    public static void main(String[] args) {
        new Calculator();
    }
}
```

Exercise 3: Modify this calculator to include subtraction, multiplication, and division buttons.

3.4. Advanced GUI Components

3.4.1. JComboBox (Dropdown Menu)

A JComboBox allows users to select from a drop-down list.

```
String[] items = {"Option 1", "Option 2", "Option 3"};
JComboBox<String> comboBox = new JComboBox<>(items);
add(comboBox);
```

3.4.2. JCheckBox

A JCheckBox is a checkable box that can be selected or deselected.

```
JCheckBox checkBox = new JCheckBox("I agree");
add(checkBox);
```

3.4.3. JRadioButton and ButtonGroup

Radio buttons allow users to select one option from a group.

```
JRadioButton option1 = new JRadioButton("Option 1");
JRadioButton option2 = new JRadioButton("Option 2");
ButtonGroup group = new ButtonGroup();
group.add(option1);
group.add(option2);
add(option1);
add(option2);
```

Exercise 4: Create a simple survey form using JTextField, JComboBox, JCheckBox, and JRadioButton. The form should have fields for name, age, gender, and a list of interests (checkable).

3.4.4. JTabbedPane

The JTabbedPane component lets you create a tabbed pane, where multiple panels are shown as tabs. This is useful for organizing content into separate sections within the same window.

Example:

```
import javax.swing.*;
public class TabbedPaneExample extends JFrame {
    public TabbedPaneExample() {
        setTitle("JTabbedPane Example");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JTabbedPane tabbedPane = new JTabbedPane();
        JPanel tab1 = new JPanel();
        tab1.add(new JLabel("Content for Tab 1"));
        JPanel tab2 = new JPanel();
        tab2.add(new JLabel("Content for Tab 2"));
        tabbedPane.addTab("Tab 1", tab1);
        tabbedPane.addTab("Tab 2", tab2)
```

```
        add(tabbedPane);
        setVisible(true);
    }
    public static void main(String[] args) {
        new TabbedPaneExample();
    }
}
```

3.4.5. JTable

The JTable component provides a powerful way to display and manage tabular data. You can display data from arrays or databases, and JTable supports features like sorting, column reordering, and cell editing.

Example:

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
public class TableExample extends JFrame {
    public TableExample() {
      setTitle("JTable Example");
       setSize(500, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        String[] columnNames = {"ID", "Name", "Age"};
        Object[][] data = {
            {1, "Alice", 23},
            {2, "Bob", 30},
            {3, "Charlie", 25}
        };
        JTable table = new JTable(data, columnNames);
        JScrollPane scrollPane = new JScrollPane(table);
        add(scrollPane);
        setVisible(true);
 }
    public static void main(String[] args) {
```

```
        new TableExample();
    }
}
```

3.4.6. JTree

The JTree component displays a hierarchical structure of nodes, like a directory structure. Each node can contain child nodes, allowing nested items.

Example:

```
import javax.swing.*;
import javax.swing.tree.DefaultMutableTreeNode;
public class TreeExample extends JFrame {
    public TreeExample() {
     setTitle("JTree Example");
      setSize(300, 400);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        DefaultMutableTreeNode root = new DefaultMutableTreeNode("Root");
        DefaultMutableTreeNode nodeA = new DefaultMutableTreeNode("Node A");
        DefaultMutableTreeNode nodeB = new DefaultMutableTreeNode("Node B");
        root.add(nodeA);
        root.add(nodeB);
        nodeA.add(new DefaultMutableTreeNode("Leaf A1"));
        nodeA.add(new DefaultMutableTreeNode("Leaf A2"));
        JTree tree = new JTree(root);
        JScrollPane scrollPane = new JScrollPane(tree);
        add(scrollPane);
        setVisible(true);
    }
    public static void main(String[] args) {
        new TreeExample();
    }
}
```

3.4.7. JProgressBar

The JProgressBar component visually indicates the progress of a task, useful for long-running processes like file downloads or loading data.

Example:

```
import javax.swing.*;
public class ProgressBarExample extends JFrame {
    JProgressBar progressBar;
    public ProgressBarExample() {
        setTitle("JProgressBar Example");
        setSize(400, 100);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        progressBar = new JProgressBar(0, 100);
        progressBar.setValue(0);
        progressBar.setStringPainted(true);
        add(progressBar);
        setVisible(true);
        // Simulating progress update
        for (int i = 0; i <= 100; i++) {
            try {
                Thread.sleep(50);
                progressBar.setValue(i);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
    public static void main(String[] args) {
        new ProgressBarExample();
    }
}
```

3.4.8. JSlider

The JSlider component is a control that allows users to select a value by moving an indicator within a bounded interval.

Example:

```
import javax.swing.*;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;
public class SliderExample extends JFrame implements ChangeListener {
   JSlider slider;
   JLabel label;
   public SliderExample() {
      setTitle("JSlider Example");
      setSize(400, 200);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      slider = new JSlider(0, 100, 50);
      slider.setMajorTickSpacing(10);
      slider.setPaintTicks(true);
      slider.setPaintLabels(true);
      slider.addChangeListener(this);
      label = new JLabel("Value: 50");
      add(slider, "North");
      add(label, "South");
      setVisible(true);
   }
   public void stateChanged(ChangeEvent e) {
      label.setText("Value: " + slider.getValue());
   }
   public static void main(String[] args) {
      new SliderExample();
   }
}
```

3.4.9. JOptionPane

The JOptionPane is a useful utility for displaying dialog boxes. You can use it for input dialogs, confirmation dialogs, and message dialogs.

Example: Message Dialog

```
import javax.swing.*;
public class OptionPaneExample {
public static void main(String[] args) {
  JOptionPane.showMessageDialog(null, "This is an informational message", "Info",
JOptionPane.INFORMATION_MESSAGE);
    }
}
```

3.4.10 JFileChooser

The JFileChooser allows users to select files or directories from their filesystem.

Example: File Chooser

```
import javax.swing.*;
import java.io.File;
public class FileChooserExample {
  public static void main(String[] args) {
   JFileChooser fileChooser = new JFileChooser();
    int option = fileChooser.showOpenDialog(null);
     if (option == JFileChooser.APPROVE_OPTION) {
          File file = fileChooser.getSelectedFile();
          System.out.println("Selected file: " + file.getAbsolutePath());
        }
    }
}
```

3.4.11. JColorChooser

The JColorChooser component allows users to select a color from a palette.

Example: Color Chooser

```
import javax.swing.*;
import java.awt.*;
public class ColorChooserExample {
    public static void main(String[] args) {
     Color color = JColorChooser.showDialog(null, "Pick a Color", Color.RED);
```

```
            if (color != null) {
                System.out.println("Selected color: " + color.toString());
            }
        }
    }
```

3.4.12. JSpinner

The JSpinner component lets users select a value from a sequence of values, such as dates or numbers.

Example: Spinner

```
import javax.swing.*;
public class SpinnerExample extends JFrame {
    public SpinnerExample() {
        setTitle("JSpinner Example");
        setSize(300, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        SpinnerNumberModel model = new SpinnerNumberModel(5, 0, 10, 1);
        JSpinner spinner = new JSpinner(model);
        add(spinner);
        setVisible(true);
    }
    public static void main(String[] args) {
        new SpinnerExample();
    }
}
```

Summary

These advanced components provide powerful ways to build sophisticated GUIs. Combined with event handling, you can create interactive and user-friendly applications.

3.5. JavaFX Layout Components

Objective:

- To learn the different layout components in JavaFX.

- To create and manage UI layouts using containers like HBox, VBox, BorderPane, and more.
- To understand nesting layouts and aligning components.

Topics Covered:

1. Setting Up JavaFX
2. Introduction to JavaFX Layout Components
3. Using Different Layouts with Examples
4. Nesting Layouts
5. Exercises

3.5.1. Setting up JavaFX

Before starting with JavaFX layouts, ensure your environment is ready to compile and run JavaFX applications. Import necessary JavaFX libraries (like javafx.application.Application, javafx.stage.Stage, and javafx.scene.Scene

3.5.2. Introduction toJavaFXLayout Components

JavaFX provides several layout panes, each with different ways of arranging child nodes. Some commonly used layout panes include:

- **HBox**: Horizontal box layout
- **VBox**: Vertical box layout
- **BorderPane**: Border layout with top, bottom, left, right, and center regions
- **GridPane**: Grid layout with rows and columns
- **StackPane**: Layers nodes on top of each other

3.5.3. Using Different Layouts with Examples

Example 1: HBox Layout

HBox arranges nodes horizontally in a single row.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.HBox;
import javafx.stage.Stage;
public class HBoxExample extends Application {
```

```
    @Override
    public void start(Stage primaryStage) {
        HBox hbox = new HBox(10);  // Spacing between nodes is 10px
        Button button1 = new Button("Button 1");
        Button button2 = new Button("Button 2");
        Button button3 = new Button("Button 3");
        hbox.getChildren().addAll(button1, button2, button3);
        Scene scene = new Scene(hbox, 300, 200);
        primaryStage.setTitle("HBox Layout Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Example 2: VBox Layout

VBox arranges nodes vertically in a single column.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
public class VBoxExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        VBox vbox = new VBox(15);  // Spacing of 15px between nodes
        Label label = new Label("VBox Layout Example");
        Button button = new Button("Click Me");
```

```
            vbox.getChildren().addAll(label, button);
            Scene scene = new Scene(vbox, 200, 150);
            primaryStage.setTitle("VBox Layout Example");
            primaryStage.setScene(scene);
            primaryStage.show();
        }

        public static void main(String[] args) {

       launch(args);

    }

}
```

Example 3: BorderPane Layout

BorderPane arranges nodes in five areas: top, bottom, left, right, and center.

```
 import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.BorderPane;
import javafx.stage.Stage;
public class BorderPaneExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        BorderPane borderPane = new BorderPane();
        Button top = new Button("Top");
        Button bottom = new Button("Bottom");
        Button left = new Button("Left");
        Button right = new Button("Right");
        Button center = new Button("Center");
        borderPane.setTop(top);
        borderPane.setBottom(bottom);
```

```
        borderPane.setLeft(left);
        borderPane.setRight(right);
        borderPane.setCenter(center);
        Scene scene = new Scene(borderPane, 300, 200);
        primaryStage.setTitle("BorderPane Layout Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Example 4: GridPane Layout

GridPane arranges nodes in a flexible grid of rows and columns.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;
public class GridPaneExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        GridPane gridPane = new GridPane();
        Button button1 = new Button("Button 1");
        Button button2 = new Button("Button 2");
        Button button3 = new Button("Button 3");
        Button button4 = new Button("Button 4");
        gridPane.add(button1, 0, 0);  // Column 0, Row 0
        gridPane.add(button2, 1, 0);  // Column 1, Row 0
        gridPane.add(button3, 0, 1);  // Column 0, Row 1
        gridPane.add(button4, 1, 1);  // Column 1, Row 1
```

```
        Scene scene = new Scene(gridPane, 300, 200);
        primaryStage.setTitle("GridPane Layout Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Example 5: StackPane Layout

StackPane places nodes on top of each other, with the last added node displayed at the top.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;
public class StackPaneExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        StackPane stackPane = new StackPane();
        Button button1 = new Button("Button 1");
        Button button2 = new Button("Button 2");
        stackPane.getChildren().addAll(button1, button2);
        Scene scene = new Scene(stackPane, 300, 200);
        primaryStage.setTitle("StackPane Layout Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
```

```
}
```

3.5.4. Nesting Layouts

You can create complex UIs by nesting layout panes within each other.

Example: Nested Layouts with HBox and VBox

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
public class NestedLayoutExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        HBox hbox = new HBox(20);  // Horizontal container
        VBox vbox = new VBox(10);  // Vertical container within HBox
        Button button1 = new Button("Button 1");
        Button button2 = new Button("Button 2");
        Button button3 = new Button("Button 3");
        vbox.getChildren().addAll(button1, button2);
        hbox.getChildren().addAll(vbox, button3);
        Scene scene = new Scene(hbox, 300, 200);
        primaryStage.setTitle("Nested Layout Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Exercises

Exercise 1: Create a login form using VBox layout. The form should contain:

- A Label for "Username"
- A TextField for username input
- A Label for "Password"
- A PasswordField for password input
- A Button labeled "Login"

Exercise 2: Using BorderPane, create a layout with navigation buttons on the left (labeled "Home", "Profile", "Settings"), a header at the top with the title "Dashboard", and an area in the center for content.

Exercise 3: Build a calculator UI using GridPane. Arrange number buttons (0-9) in a grid, along with operation buttons (+, -, *, /) and an "=" button at the bottom.

Summary

- JavaFX layout panes provide flexible ways to arrange components.
- Understanding the different layouts (HBox, VBox, BorderPane, etc.) helps in designing structured UIs.
- Nesting layout panes allows for more complex UI designs.

3.6. Basic UI Controls

In Java programming, **Basic UI controls** are part of the `javax.swing` package and allow you to create graphical user interfaces (GUIs). Here's a set of lab demonstrations to help you understand and practice working with basic UI controls like buttons, labels, text fields, checkboxes, radio buttons, and combo boxes.

3.6.1. Simple Window with a Button

In this example, we will create a simple window with a button that prints a message when clicked.

```
import javax.swing.*;
import java.awt.event.*;
public class BasicUIControls {
    public static void main(String[] args) {
        // Create a JFrame (main window)
```

```
        JFrame frame = new JFrame("Basic UI Controls");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(300, 200);
        // Create a JButton
        JButton button = new JButton("Click Me!");
        // Add an action listener to the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                JOptionPane.showMessageDialog(frame, "Button clicked!");
            }
        });
        // Add the button to the frame
        frame.getContentPane().add(button);
        // Set frame visible
        frame.setVisible(true);
    }
}
```

3.6.2. Text Field and Label Interaction

This example demonstrates a text field where you can enter text, and a label will display the entered text when a button is clicked.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class TextFieldDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Text Field and Label Interaction");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 200);
        frame.setLayout(new FlowLayout());
        // Create a label and text field
        JLabel label = new JLabel("Enter text:");
```

```
        JTextField textField = new JTextField(20);
        JButton button = new JButton("Show Text");
        // Action listener for the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String text = textField.getText();
                label.setText("Entered text: " + text);
            }
        });
        // Add components to the frame
        frame.add(label);
        frame.add(textField);
        frame.add(button);
        frame.setVisible(true);
    }
}
```

3.6.3. Checkboxes

This example demonstrates how to work with checkboxes.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class CheckboxDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Checkbox Demo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 200);
        frame.setLayout(new FlowLayout());
        // Create checkboxes
        JCheckBox checkBox1 = new JCheckBox("Option 1");
        JCheckBox checkBox2 = new JCheckBox("Option 2");
        // Create a button to display selected checkboxes
```

```
        JButton button = new JButton("Show Selected Options");
        // Action listener for the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String selectedOptions = "Selected: ";
                if (checkBox1.isSelected()) {
                    selectedOptions += "Option 1 ";
                }
                if (checkBox2.isSelected()) {
                    selectedOptions += "Option 2 ";
                }
                JOptionPane.showMessageDialog(frame, selectedOptions);
            }
        });
        // Add components to the frame
        frame.add(checkBox1);
        frame.add(checkBox2);
        frame.add(button);
        frame.setVisible(true);
    }
}
```

3.6.4. Radio Buttons

Radio buttons allow the user to select one option from a group. This example demonstrates how to handle radio buttons.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class RadioButtonDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Radio Button Demo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
frame.setSize(400, 200);
frame.setLayout(new FlowLayout());
// Create radio buttons
JRadioButton radioButton1 = new JRadioButton("Option 1");
JRadioButton radioButton2 = new JRadioButton("Option 2");
JRadioButton radioButton3 = new JRadioButton("Option 3");
// Group the radio buttons
ButtonGroup group = new ButtonGroup();
group.add(radioButton1);
group.add(radioButton2);
group.add(radioButton3);
// Create a button to display selected radio button
JButton button = new JButton("Show Selected Option");
// Action listener for the button
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (radioButton1.isSelected()) {
            JOptionPane.showMessageDialog(frame, "Option 1 selected");
        } else if (radioButton2.isSelected()) {
            JOptionPane.showMessageDialog(frame, "Option 2 selected");
        } else if (radioButton3.isSelected()) {
            JOptionPane.showMessageDialog(frame, "Option 3 selected");
        } else {
            JOptionPane.showMessageDialog(frame, "No option selected");
        }
    }
});
// Add components to the frame
frame.add(radioButton1);
frame.add(radioButton2);
frame.add(radioButton3);
```

```
        frame.add(button);
        frame.setVisible(true);
    }
}
```

3.6.5. Combo Box

A combo box allows the user to choose from a drop-down list of options.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class ComboBoxDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Combo Box Demo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 200);
        frame.setLayout(new FlowLayout());
        // Create combo box with options
        String[] options = { "Option 1", "Option 2", "Option 3", "Option 4" };
        JComboBox<String> comboBox = new JComboBox<>(options);
        // Create a button to show selected combo box option
        JButton button = new JButton("Show Selected Option");
        // Action listener for the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String selectedOption = (String) comboBox.getSelectedItem();
                JOptionPane.showMessageDialog(frame, "Selected: " + selectedOption);
            }
        });
        // Add components to the frame
        frame.add(comboBox);
        frame.add(button);
        frame.setVisible(true);
```

```
        }
    }
```

Summary:

These demonstrations cover the following basic UI controls in Java:

- **Button**: Respond to clicks.
- **Text Field**: Allows user input.
- **Label**: Displays static text.
- **Checkbox**: Allows selecting multiple options.
- **Radio Button**: Allows selecting one option from a set.
- **Combo Box**: Provides a drop-down list of choices.

You can experiment with these examples, modifying them to explore more features and create more complex interfaces.

3.7. Event Handlers in Java Programming

In Java programming, **event handling** refers to the process by which a program responds to user interactions with the graphical user interface (GUI), such as button clicks, mouse movements, or key presses. Event handlers are methods that handle these events. In Java, event handling is typically done through the **ActionListener**, **MouseListener**, **KeyListener,** and other listener interfaces, all part of the **java.awt.event** package.

This lab manual will guide you through creating and using various event handlers in Java using Swing components.

Objective:

- To understand and implement basic event handling in Java using Swing components and different types of event listeners.

Prerequisites:

- Knowledge of Java programming.
- Understanding of Swing components like buttons, text fields, labels, etc.

3.7.1. ActionEvent - Button Click

In this lab, we will create a simple program with a button that responds to the click event.

Objective: Handle a button click event using the **ActionListener** interface.

```
mport javax.swing.*;
import java.awt.event.*;
public class ActionEventDemo {
    public static void main(String[] args) {
        // Cireate the frame
        JFrame frame = new JFrame("Button Click Event");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Create a button
        JButton button = new JButton("Click Me!");
        // Add an ActionListener to the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                JOptionPane.showMessageDialog(frame, "Button clicked!");
            }
        });
        // Add the button to the frame
        frame.add(button);
        // Display the frame
        frame.setVisible(true);
    }
}
```

Steps:

1. Create a **JButton** to trigger an event.
2. Implement the **ActionListener** interface to listen for button click events.
3. Use the **actionPerformed** method to define the behavior when the button is clicked.

3.7.2. MouseEvent - Mouse Click

This lab will teach you how to handle mouse events like clicking and entering a component.

Objective: Handle mouse click events using the **MouseListener** interface.

```
import javax.swing.*;
import java.awt.event.*;
```

```
public class MouseEventDemo {
    public static void main(String[] args) {
        // Create the frame
        JFrame frame = new JFrame("Mouse Click Event");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Create a label
        JLabel label = new JLabel("Click anywhere in the frame");
        // Add a MouseListener to the label
        label.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
 JOptionPane.showMessageDialog(frame, "Mouse clicked at (" + e.getX() + ", " +
e.getY() + ")");
            }
        });
        // Add the label to the frame
        frame.add(label);
        // Display the frame
        frame.setVisible(true);
    }
}
```

Steps:

1. Create a **JLabel** or **JPanel** where you will detect the mouse click.
2. Use the **MouseAdapter** class to implement the `MouseListener` interface for simplicity.
3. Override the **mouseClicked** method to define the behavior on mouse clicks.

3.7.3. KeyEvent - Key Press

This lab will handle keyboard input and detect key presses in a text field.

Objective: Handle key press events using the **KeyListener** interface.

```
import javax.swing.*;
import java.awt.event.*;
```

```
public class KeyEventDemo {
    public static void main(String[] args) {
        // Create the frame
        JFrame frame = new JFrame("Key Press Event");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Create a text field
        JTextField textField = new JTextField(20);
        // Add a KeyListener to the text field
        textField.addKeyListener(new KeyAdapter() {
            public void keyPressed(KeyEvent e) {
                char key = e.getKeyChar();
                JOptionPane.showMessageDialog(frame, "Key pressed: " + key);
            }
        });
        // Add the text field to the frame
        frame.add(textField);
        // Display the frame
        frame.setVisible(true);
    }
}
```

Steps:

1. Create a **JTextField** to accept keyboard input.
2. Use the **KeyAdapter** class to implement the **KeyListener** interface.
3. Override the **keyPressed** method to capture the key that was pressed.

3.7.4. WindowEvent - Window Closing

This lab demonstrates handling window close events, such as when the user clicks the close button.

Objective: Handle window closing events using the `WindowListener` interface.

```
import javax.swing.*;
```

```
import java.awt.event.*;
public class WindowEventDemo {
    public static void main(String[] args) {
        // Create the frame
        JFrame frame = new JFrame("Window Closing Event");
        frame.setSize(300, 200);
        // Add a WindowListener to the frame
        frame.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
    int response = JOptionPane.showConfirmDialog(frame, "Are you sure you want to
exit?");
                if (response == JOptionPane.YES_OPTION) {
                    System.exit(0);
                }
            }
        });
        // Display the frame
        frame.setVisible(true);
    }
}
```

Steps:

1. Use a **WindowListener** to handle window events.
2. Override the **windowClosing** method to prompt the user before exiting the application.

3.7.5. ItemEvent - ComboBox Selection

In this lab, we will handle events when an item is selected from a combo box.

Objective: Handle selection changes in a combo box using the **ItemListener** interface.

```
import javax.swing.*;
import java.awt.event.*;
public class ItemEventDemo {
    public static void main(String[] args) {
```

```
        // Create the frame
        JFrame frame = new JFrame("ComboBox Item Selection");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Create a combo box with options
        String[] options = { "Option 1", "Option 2", "Option 3" };
        JComboBox<String> comboBox = new JComboBox<>(options);
        // Add an ItemListener to the combo box
        comboBox.addItemListener(new ItemListener() {
            public void itemStateChanged(ItemEvent e) {
                if (e.getStateChange() == ItemEvent.SELECTED) {
                    String selected = (String) e.getItem();
                    JOptionPane.showMessageDialog(frame, "You selected: " + selected);
                }
            }
        });
        // Add the combo box to the frame
        frame.add(comboBox);
        // Display the frame
        frame.setVisible(true);
    }
}
```

Steps:

1. Create a **JComboBox** with a list of items.
2. Use an **ItemListener** to detect changes in the selected item.
3. Override the **itemStateChanged** method to display a message when an item is selected.

Summary of Event Handlers:

- **ActionListener:** For responding to button clicks and other actions like menu selections.
- **MouseListener:** For detecting mouse events (click, enter, exit, etc.).

- **KeyListener:** For detecting keyboard input.
- **WindowListener:** For handling window events like closing or opening a window.
- **ItemListener:** For handling item selection changes in combo boxes, checkboxes, etc.

Summary:

By completing the above lab demonstrations, you will have learned the basics of **event handling** in Java. Event handlers allow you to create interactive applications where the program responds to user inputs such as mouse clicks, key presses, and other events.

3.8. UI Controls in Java Programming

In Java, **UI controls** (also known as widgets or components) are elements of the graphical user interface (GUI) that allow the user to interact with the application. These controls include buttons, text fields, labels, checkboxes, radio buttons, combo boxes, and more, all of which are part of the **javax.swing** package.

This lab manual will guide you through the usage and implementation of common UI controls in Java programming using Swing.

Objective: To learn how to use various UI controls in Java, including buttons, labels, text fields, checkboxes, radio buttons, and combo boxes, by creating interactive applications.

Prerequisites:

- Basic knowledge of Java programming.
- Familiarity with Swing components **(JFrame, JButton, JLabel**, **JTextField**, etc.).
- Java Development Kit (JDK) installed.

3.8.1. Button Control

In this lab, you will create a simple window with a button, and you will handle its click event.

Objective: Implement a button and handle its action event using **ActionListener**.

```
import javax.swing.*;
import java.awt.event.*;
public class ButtonControl {
```

```
    public static void main(String[] args) {
        // Create a frame
        JFrame frame = new JFrame("Button Control");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Create a button
        JButton button = new JButton("Click Me");
        // Add an ActionListener to the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                JOptionPane.showMessageDialog(frame, "Button clicked!");
            }
        });
        // Add the button to the frame
        frame.add(button);
        // Set frame visible
        frame.setVisible(true);
    }
}
```

Steps:

1. Create a **JButton**.
2. Add an **ActionListener** to handle button clicks.
3. Show a dialog when the button is clicked.

3.9.2. Label Control

In this lab, we will use a label to display text on the window.

Objective: Implement a `JLabel` and modify its text programmatically.

```
import javax.swing.*;
public class LabelControl {
    public static void main(String[] args) {
        // Create a frame
        JFrame frame = new JFrame("Label Control");
```

```
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Create a label
        JLabel label = new JLabel("Hello, this is a label");
        // Set label's alignment
        label.setHorizontalAlignment(SwingConstants.CENTER);
        // Add the label to the frame
        frame.add(label);
        // Set frame visible
        frame.setVisible(true);
    }
}
```

Steps:

1. Create a **JLabel** to display static text.
2. Set the label's alignment to center.
3. Add the label to the frame and display it.

3.8.2. Text Field Control

In this lab, you will work with a text field, allowing users to enter text.

Objective: Implement a **JTextField** for user input and capture the text entered.

```
import javax.swing.*;
import java.awt.event.*;
public class TextFieldControl {
    public static void main(String[] args) {
        // Create a frame
        JFrame frame = new JFrame("Text Field Control");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Create a text field
        JTextField textField = new JTextField(20);
        // Create a button
        JButton button = new JButton("Show Text");
```

```
        // Add an ActionListener to the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String text = textField.getText();
                JOptionPane.showMessageDialog(frame, "You entered: " + text);
            }
        });
        // Add components to the frame
        frame.setLayout(new FlowLayout());
        frame.add(textField);
        frame.add(button);
        // Set frame visible
        frame.setVisible(true);
    }
}
```

Steps:

1. Create a **JTextField** for input.
2. Use a button to display the text entered in the text field.
3. Handle the button click using an **ActionListener**.

3.8.3. CheckBox Control

In this lab, you will work with checkboxes, allowing users to select or deselect options.

Objective: Implement a `JCheckBox` and capture its state (selected or deselected).

```
import javax.swing.*;
import java.awt.event.*;
public class CheckBoxControl {
    public static void main(String[] args) {
        // Create a frame
        JFrame frame = new JFrame("CheckBox Control");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        // Create checkboxes
        JCheckBox checkBox1 = new JCheckBox("Option 1");
        JCheckBox checkBox2 = new JCheckBox("Option 2");
        // Create a button to show selected checkboxes
        JButton button = new JButton("Show Selected");
        // Add an ActionListener to the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String selectedOptions = "Selected options: ";
                if (checkBox1.isSelected()) {
                    selectedOptions += "Option 1 ";
                }
                if (checkBox2.isSelected()) {
                    selectedOptions += "Option 2 ";
                }
                JOptionPane.showMessageDialog(frame, selectedOptions);
            }
        });
        // Add components to the frame
        frame.setLayout(new FlowLayout());
        frame.add(checkBox1);
        frame.add(checkBox2);
        frame.add(button);
        // Set frame visible
        frame.setVisible(true);
    }
}
```

Steps:

1. Create **JCheckBox** components for multiple selections.
2. Use a button to display the selected options.
3. Handle the checkbox state using the isSelected() method.

3.8.4. Radio Button Control

In this lab, you will work with radio buttons, which allow users to choose one option from a group.

Objective: Implement **JRadioButton** and group them so that only one can be selected at a time.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class RadioButtonControl {
   public static void main(String[] args) {
            // Create a frame
      JFrame frame = new JFrame("Radio Button Control");
      frame.setSize(300, 200);
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            // Create radio buttons
      JRadioButton radioButton1 = new JRadioButton("Option 1");
      JRadioButton radioButton2 = new JRadioButton("Option 2");
            // Group radio buttons together
      ButtonGroup group = new ButtonGroup();
      group.add(radioButton1);
      group.add(radioButton2);
            // Create a button to display the selected radio button
      JButton button = new JButton("Show Selected");
            // Add an ActionListener to the button
      button.addActionListener(new ActionListener() {
         public void actionPerformed(ActionEvent e) {
            if (radioButton1.isSelected()) {
               JOptionPane.showMessageDialog(frame, "Option 1 selected");
            } else if (radioButton2.isSelected()) {
               JOptionPane.showMessageDialog(frame, "Option 2 selected");
            } else {
               JOptionPane.showMessageDialog(frame, "No option selected");
            }
         }
      });
         // Add components to the frame
      frame.setLayout(new FlowLayout());
```

```
        frame.add(radioButton1);
        frame.add(radioButton2);
        frame.add(button);
          // Set frame visible
        frame.setVisible(true);
    }
}
```

Steps:

1. Create **JRadioButton** components.
2. Group the radio buttons using a `ButtonGroup` so only one can be selected at a time.
3. Use a button to display the selected option.

3.8.5. Combo Box Control

In this lab, you will work with a combo box, which displays a drop-down list of options.

Objective: Implement a `JComboBox` and capture the selected item.

```
import javax.swing.*;
import java.awt.event.*;
public class ComboBoxControl {
    public static void main(String[] args) {
        // Create a frame

        JFrame frame = new JFrame("Combo Box Control");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Create a combo box with options
        String[] options = { "Option 1", "Option 2", "Option 3" };
        JComboBox<String> comboBox = new JComboBox<>(options);
        // Create a button to show the selected option
        JButton button = new JButton("Show Selected");
        // Add an ActionListener to the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String selectedOption = (String) comboBox.getSelectedItem();
             JOptionPane.showMessageDialog(frame, "You selected: " + selectedOption);
            }
        });
        // Add components to the frame
        frame.setLayout(new FlowLayout());
        frame.add(comboBox);
        frame.add(button);
        // Set frame visible
```

```
            frame.setVisible(true);
        }
    }
```

Steps:

1. Create a **JComboBox** with a list of options.
2. Use a button to display the selected option.
3. Capture the selected option with **getSelectedItem()**.

3.9. Working with Shapes

Objective:

- To learn how to create and manipulate basic shapes (circles, rectangles, lines) using Java's 2D graphics library.
- To understand how to use graphics and paint components to render shapes on a GUI.

Prerequisites:

- Basic knowledge of Java programming.
- Familiarity with Java Swing and AWT (Abstract Window Toolkit).

3.9.1. Introduction to Java 2D Graphics

Java provides a rich set of libraries for creating and manipulating graphics, most notably in the **java.awt** and **javax.swing** packages. The **Graphics** and **Graphics2D** classes provide the tools to draw shapes, lines, and text on the screen.

Setting Up the Lab Environment

1. Install the JDK on your computer.
2. . Set up a new Java project in your chosen IDE.
3. Create a new Java class to begin the exercises (e.g., `ShapesDemo`).

3.9.2. Drawing Basic Shapes: Using Graphics

In this section, you'll learn how to draw basic shapes such as lines, rectangles, and circles using the **Graphics** object.

Task 1: Drawing a Line

Create a simple Java class to draw a line.

```
import javax.swing.*;
import java.awt.*;
public class ShapesDemo extends JPanel {
```

```
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        // Cast Graphics to Graphics2D for more advanced features
        Graphics2D g2d = (Graphics2D) g;
        // Drawing a line from (50, 50) to (200, 200)
        g2d.drawLine(50, 50, 200, 200);
    }
    public static void main(String[] args) {
        JFrame frame = new JFrame();
        ShapesDemo panel = new ShapesDemo();
        frame.add(panel);
        frame.setSize(400, 400);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Explanation:

- **paintComponent(Graphics g)**: This method is overridden to customize the drawing behavior. It's called automatically by Swing when the component needs to be rendered.
- **drawLine(int x1, int y1, int x2, int y2)**: Draws a line between two points.

Task 2: Drawing a Rectangle

```
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;
        // Draw a rectangle (x, y, width, height)
        g2d.drawRect(50, 50, 200, 100);
    }
```

Task 3: Drawing an Ellipse

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2d = (Graphics2D) g;
    // Draw an ellipse inside the rectangle (x, y, width, height)
    g2d.drawOval(50, 50, 200, 100);
}
```

Task 4: Drawing Filled Shapes

To create a filled shape, we use the **fillRect** or **fillOval** methods instead of **drawRect** or **drawOval**.

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2d = (Graphics2D) g;
    // Draw a filled rectangle
    g2d.setColor(Color.RED);  // Set color to red
    g2d.fillRect(50, 50, 200, 100); // Fill the rectangle with red
    // Draw a filled ellipse
    g2d.setColor(Color.BLUE);  // Set color to blue
    g2d.fillOval(50, 200, 200, 100); // Fill the ellipse with blue
}
```

Explanation:

- **fillRect(int x, int y, int width, int height):** Fills a rectangle with the current drawing color.
- **fillOval(int x, int y, int width, int height):** Fills an ellipse with the current drawing color.

3.9.3. Transformations and Rotation

Java's **Graphics2D** class allows you to perform transformations such as rotation, scaling, and translation on shapes.

Task 5: Rotating a Shape

To rotate a shape, you can use the **rotate()** method of **Graphics2D.**

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2d = (Graphics2D) g;
    // Rotate the drawing by 45 degrees
    g2d.rotate(Math.toRadians(45), 150, 150);  // Rotate around the center (150, 150)
    // Draw a rectangle after rotation
    g2d.setColor(Color.GREEN);
    g2d.fillRect(100, 100, 100, 50);
}
```

Explanation:

- **rotate(double theta, double x, double y):** Rotates the drawing by **theta** degrees around the point **(x, y).**

3.9.4. Adding Stroke and Color to Shapes

You can modify the thickness and color of the shape's outline by changing the **stroke** and **color** properties.

Task 6: Customizing the Stroke and Color

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2d = (Graphics2D) g;
    // Set stroke thickness
    g2d.setStroke(new BasicStroke(5));  // 5-pixel thick line
    // Set color for the outline
    g2d.setColor(Color.BLACK);
    g2d.drawRect(50, 50, 200, 100);  // Rectangle outline
    // Set color for filling
    g2d.setColor(Color.YELLOW);
    g2d.fillRect(50, 200, 200, 100);  // Filled rectangle
}
```

Explanation:

- **setStroke(Stroke s):** Sets the thickness of the outline.
- **setColor(Color c):** Sets the color used for the shapes.

3.9.5. Handling Mouse Events with Shapes

You can also allow users to interact with shapes by capturing mouse events.

Task 7: Drawing Shapes on Mouse Click

```
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
public class ShapesDemo extends JPanel {
    private int x, y;
    public ShapesDemo() {
        addMouseListener(new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                x = e.getX();
                y = e.getY();
                repaint();  // Repaint to draw the circle at the new location
            }
        });
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;
        g2d.setColor(Color.RED);
        g2d.fillOval(x - 25, y - 25, 50, 50);  // Draw a circle at mouse location
    }
    public static void main(String[] args) {
        JFrame frame = new JFrame();
        ShapesDemo panel = new ShapesDemo();
        frame.add(panel);
        frame.setSize(400, 400);
```

```
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setVisible(true);
        }
    }
```

Explanation:

- **MouseListener**: Captures mouse events like clicks, presses, and releases.
- **getX()** and **getY():** Retrieve the mouse coordinates when clicked.

Summary

In this lab, you learned how to create basic shapes like lines, rectangles, and ellipses, and how to manipulate those using transformations. You also explored customizing shapes with colors and strokes and handled user interactions using mouse events.

3.10. CSS Styling in Java Programming (JavaFX)

Objective:

- To learn how to apply CSS styling to JavaFX applications.
- To understand how to customize UI components in JavaFX using CSS for better appearance and layout control.

Prerequisites:

- Basic knowledge of Java programming.
- Familiarity with JavaFX for creating graphical user interfaces.
- Understanding of basic CSS (styles for fonts, colors, margins, etc.).

3.10.1. Introduction to JavaFX and CSS

JavaFX allows you to build rich and interactive graphical user interfaces in Java. Similar to web development where HTML elements are styled using CSS, JavaFX also supports styling its UI components via an external or embedded CSS file.

Key Concepts:

- **CSS for JavaFX**: Used to style JavaFX components such as buttons, labels, and containers.
- **Selector Types:** Like in web development, CSS in JavaFX supports class, ID, and type selectors.
- **JavaFX CSS Properties:** JavaFX uses CSS properties for styling elements, such as **background-color**, **font-size, border**, and more.

3.10.2. Basic JavaFX Program and Styling with CSS

Start by creating a basic JavaFX application that will utilize an external CSS file to style the user interface components.

Task 1: Basic JavaFX Application Setup

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;
public class CSSStylingDemo extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button btn = new Button("Click Me");
        btn.setId("myButton");  // Setting an ID for CSS styling
        StackPane root = new StackPane();
        root.getChildren().add(btn);
        Scene scene = new Scene(root, 300, 250);
        scene.getStylesheets().add("style.css");  // Adding the external CSS file
        primaryStage.setTitle("JavaFX CSS Styling");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Explanation:

- The **Button** is created with the text "**Click Me**".
- The **setId("myButton")** method is used to assign an ID to the button so it can be styled via CSS.
- The CSS file **style.css** is applied using **scene.getStylesheets().add("style.css");.**

3.10.3. Styling the JavaFX Components using CSS

Now, let's create an external **CSS file** (**style.css**) to style the components in the JavaFX application.

Task 2: CSS Styling

```
/* style.css */
#myButton {
    -fx-background-color: #4CAF50;  /* Green background */
    -fx-text-fill: white;           /* White text */
    -fx-font-size: 16px;            /* Font size */
    -fx-padding: 10px 20px;         /* Padding around text */
    -fx-border-radius: 5px;         /* Rounded corners */
}
.root {
    -fx-background-color: #f0f0f0;   /* Light gray background for the root */
    -fx-alignment: center;           /* Center the content */
}
```

Explanation:

- **#myButton:** This is the ID selector for the button with the ID **myButton**. It styles the background color, font size, text color, padding, and border radius.
- .root: The root element (in this case, `StackPane`) is given a light gray background and the content is centered.

Running the Program

- Compile and run the program.
- The button should have a green background, white text, and rounded corners, styled according to the CSS.

3.10.4. Advanced CSS Styling Techniques in JavaFX

Now, let's explore more advanced features of CSS styling in JavaFX, such as modifying fonts, borders, and handling different UI states.

Task 3: Changing Font Style and Borders

```
/* Style for Button with hover effect */
#myButton {
```

```
    -fx-background-color: #4CAF50;
    -fx-text-fill: white;
    -fx-font-size: 16px;

   -fx-font-family: "Arial";      /* Custom font */
    -fx-padding: 10px 20px;
    -fx-border-radius: 5px;
    -fx-border-color: black;     /* Black border */
    -fx-border-width: 2px;       /* Border width */
}
/* Hover effect */
#myButton:hover {
    -fx-background-color: #45a049; /* Darker green on hover */
}
```

Explanation:

- **-fx-font-family:** "Arial": Changes the font of the button text to Arial.
- **-fx-border-color** and **fx-border-width**: Adds a border around the button with a black color and width of 2px.
- **#myButton:hover:** Defines the hover effect that changes the background color when the mouse hovers over the button.

3.10.5. **Using CSS for Layouts and Effects**

CSS can also be used to modify the layout and visual effects of JavaFX components.

Task 4: Applying Effects and Layout Styling

```
/* Styling for root layout */
.root {
    -fx-background-color: #f0f0f0;
    -fx-alignment: center;
    -fx-spacing: 20px;             /* Adds spacing between components */
}
/* Drop shadow effect for the button */
#myButton {
    -fx-background-color: #4CAF50;
    -fx-text-fill: white;
    -fx-font-size: 16px;
    -fx-padding: 10px 20px;
    -fx-border-radius: 5px;
    -fx-effect: dropshadow(gaussian, #000000, 10, 0, 2, 2);  /* Drop shadow effect */
}
```

Explanation:

- **-fx-spacing: 20px:** Adds spacing between components in the layout.

- **-fx-effect: dropshadow(...):** Applies a drop shadow effect to the button, making it look more 3D.

3.10.6. JavaFX CSS Pseudo-Classes

JavaFX supports pseudo-classes like **:hover**, **:focused**, and **:disabled**, which can be used to style UI elements based on their state.

Task 5: Pseudo-Classes Example

```
/* Styling for Button when focused */
#myButton:focused {
    -fx-background-color: #8BC34A;  /* Lighter green when focused */
 -fx-border-color: #388E3C;      /* Dark green border */
}
/* Styling for Button when disabled */
#myButton:disabled {
    -fx-background-color: #B0BEC5;  /* Gray color when disabled */
    -fx-text-fill: #607D8B;         /* Dark gray text */
}
```

Explanation:

- **:focused:** Applies styling when the button is focused (clicked or selected).
- **:disabled**: Applies styling when the button is disabled.

Summary

In this lab, you learned how to integrate CSS with JavaFX to style user interface components. We covered basic styling concepts, advanced styling techniques, effects, and pseudo-classes that allow for responsive design in JavaFX applications.

1. **Additional Tasks for Practice**
    - **Styling Different JavaFX Components**: Style different JavaFX components like `**TextField**`, `**Labe**l`, and `**CheckBox**` using CSS.
    - **Customizing JavaFX Themes**: Create a dark theme or light theme for your JavaFX application using CSS.
    - **Transitions and Animations**: Experiment with CSS transitions and animations to create smooth UI effects (e.g., button hover effects).

3.11. Property Bindings in Java Programming (JavaFX)

Objective:

- To understand how property bindings work in JavaFX.
- To learn how to bind properties of different JavaFX components so that they stay in sync automatically.
- To use JavaFX property bindings to improve the reactivity and maintainability of your applications.

Prerequisites:

- Basic knowledge of Java programming.
- Familiarity with JavaFX components (buttons, text fields, labels, etc.).
- Understanding of JavaFX properties, particularly StringProperty, IntegerProperty, BooleanProperty, etc.

3.11.1. Introduction to Property Bindings

JavaFX introduces a concept known as **Property Bindings**, which allows properties of UI components to be linked together so that when one property changes, the other is updated automatically.

Property: A special kind of object in JavaFX that holds a value and can be observed. For example, `StringProperty` or `IntegerProperty` is a property that wraps a `String` or `Integer`.

Binding: A mechanism that allows one property to automatically update when another property changes.

JavaFX properties can be bound in two ways:

A. **Bidirectional Binding**: Changes in one property automatically update the other.

B. **Unidirectional Binding**: One property follows changes of another, but the reverse does not occur.

Setting Up the Lab Environment

1. Install JDK (Java 8 or higher) and ensure JavaFX is included.
2. Create a New JavaFX Project in your IDE.
3. Create Java and FXML Files for the UI (optional).

3.11.2. Basic JavaFX Application Setup

In this section, you'll start by setting up a basic JavaFX application that demonstrates property binding.

Task 1: Simple Property Binding Example

Let's start by binding the text property of a Label to the text property of a TextField.

```
import javafx.application.Application;
import javafx.beans.property.StringProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
public class PropertyBindingDemo extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Creating a TextField and Label
        TextField textField = new TextField();
        Label label = new Label("Type something...");
        // Creating a StringProperty to bind
        StringProperty textProperty = new SimpleStringProperty();
        textField.textProperty().bindBidirectional(textProperty);//Binding TextField to the StringProperty
        label.textProperty().bind(textProperty);  // Binding the StringProperty to the Label's text
        VBox vbox = new VBox(textField, label);
        Scene scene = new Scene(vbox, 300, 200);
        primaryStage.setTitle("Property Binding Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
```

```
        launch(args);
    }
}
```

Explanation:

- **StringProperty textProperty = new SimpleStringProperty();:** A StringProperty is created to hold the value typed into the TextField.
- **bindBidirectional(textProperty)**: Binds the TextField's textProperty to the StringProperty. Changes in the text field update the property and vice versa.
- **label.textProperty().bind(textProperty);**: The Label's textProperty is bound to the same StringProperty. Any change to the property is reflected in the label's text.

3.11.2. Unidirectional Property Binding

In unidirectional binding, one property updates another without the reverse happening.

Task 2: Unidirectional Binding Example

In this example, we will bind the TextField to the Label's text, but the reverse will not happen.

```
import javafx.application.Application;
import javafx.beans.property.StringProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
public class UnidirectionalBindingDemo extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create the TextField and Label
        TextField textField = new TextField();
        Label label = new Label("Enter text in the box");
        // Bind the TextField's text property to the Label's text property
```

```
        label.textProperty().bind(textField.textProperty()); // Only the Label's text will
change when TextField changes
  VBox vbox = new VBox(textField, label);
      Scene scene = new Scene(vbox, 300, 200);
  primaryStage.setTitle("Unidirectional Property Binding");
      primaryStage.setScene(scene);
      primaryStage.show();
   }
   public static void main(String[] args) {
      launch(args);
   }
}
```

Explanation

- **label.textProperty().bind(textField.textProperty());:** This binds the Label's text property to the TextField's text property. Now, when the user types in the TextField, the Label's text will automatically update, but typing in the label won't change the text field.

3.11.3. Bidirectional Property Binding

Bidirectional binding allows two properties to stay in sync automatically.

Task 3: Bidirectional Property Binding Example

Let's create an example with bidirectional binding where two properties (of two TextField components) are linked.

```
import javafx.application.Application;
import javafx.beans.property.StringProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.scene.Scene;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
public class BidirectionalBindingDemo extends Application {
```

```
    @Override
    public void start(Stage primaryStage) {
        // Create two TextFields
        TextField textField1 = new TextField();
        TextField textField2 = new TextField();
        // Create StringProperties for binding
        StringProperty property1 = new SimpleStringProperty();
        StringProperty property2 = new SimpleStringProperty();
        // Bind both properties bidirectionally
        textField1.textProperty().bindBidirectional(property1);
    textField2.textProperty().bindBidirectional(property2);
        // Bind the two properties bidirectionally to each other
        property1.bindBidirectional(property2); // Changes in one TextField update the
other
        VBox vbox = new VBox(textField1, textField2);
        Scene scene = new Scene(vbox, 300, 200);
        primaryStage.setTitle("Bidirectional Property Binding");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Explanation:

- **textField1.textProperty().bindBidirectional(property1)**;: Binds the first TextField's text to the first StringProperty.
- **textField2.textProperty().bindBidirectional(property2);:** Binds the second TextField's text to the second StringProperty.

- **property1.bindBidirectional(property2);:** Bidirectionally binds both properties to each other, so when you type into one text field, the other updates automatically.

3.11.4. Advanced Property Bindings with Listeners

In some cases, you may need to listen for changes to a property and execute additional logic.

Task 4: Using a ChangeListener

In this task, we will bind a property and also use a listener to track changes.

```
import javafx.application.Application;
import javafx.beans.value.ChangeListener;
import javafx.beans.value.ObservableValue;
import javafx.beans.property.StringProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
public class PropertyChangeListenerDemo extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create TextField and Label
        TextField textField = new TextField();
        Label label = new Label("Type something:");
        // Create a StringProperty
        StringProperty textProperty = new SimpleStringProperty();
        textField.textProperty().bindBidirectional(textProperty);
        // Add a ChangeListener to the property
        textProperty.addListener(new ChangeListener<String>() {
            @Override
            public void changed(ObservableValue<? extends String> observable, String
oldValue, String newValue) {
```

```
                System.out.println("Text changed: " + newValue); // Log the change
            }
        });
        VBox vbox = new VBox(textField, label);
        Scene scene = new Scene(vbox, 300, 200);
        primaryStage.setTitle("Property Change Listener");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Explanation:

- **addListener():** Adds a listener to a property that triggers whenever the property changes.
- **ChangeListener<String>:** A listener that receives updates whenever the value of the StringProperty changes.

Summery

In this lab, you learned the concept of property bindings in JavaFX. By using property bindings, we can ensure that UI components stay in sync automatically, leading to more maintainable and reactive applications. We explored both **unidirectional** and **bidirectional bindings** and also introduced **change listeners** for more complex interactions.

3.12. Graphics and Animation in (JavaFX)

Objective:

- To understand the basics of graphics programming in Java using JavaFX.
- To learn how to create and animate graphical objects such as shapes and images.
- To implement animations using JavaFX's built-in classes and methods.

Prerequisites:

- Basic knowledge of Java programming.
- Familiarity with JavaFX (controls, layouts, etc.).

- Understanding of basic concepts in graphics (coordinates, shapes, colors).

3.12.1. Introduction to Graphics and Animation in JavaFX

JavaFX provides powerful tools to work with graphics and animations. You can draw shapes, handle mouse events, and animate objects to create interactive and visually rich applications.

- **Graphics**: JavaFX allows drawing basic shapes like rectangles, circles, lines, etc.
- **Animation**: JavaFX provides classes such as Timeline KeyFrame, and TranslateTransition to animate objects.

Setting Up the Lab Environment

1. Install JDK (Java 8 or higher) and ensure JavaFX is included.
2. Create a New JavaFX Project in your IDE.
3. Create Java files for the graphical and animation code.

3.12.2. Drawing Basic Shapes in JavaFX

Before we get into animations, let's start with drawing basic shapes.

Task 1: Drawing Basic Shapes

In this task, we will create a simple JavaFX application that draws different shapes such as a circle, rectangle, and line.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.shape.Line;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;
public class GraphicsDemo extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create a Pane to hold the shapes
        Pane pane = new Pane();
        // Create a Circle with a radius of 50
```

```
        Circle circle = new Circle(100, 100, 50);
        circle.setFill(Color.RED);
        // Create a Rectangle with width 200 and height 100
        Rectangle rectangle = new Rectangle(200, 50, 200, 100);
        rectangle.setFill(Color.BLUE);
        // Create a Line from (0, 0) to (400, 400)
        Line line = new Line(0, 0, 400, 400);
        line.setStroke(Color.GREEN);
        line.setStrokeWidth(2);
        // Add the shapes to the Pane
        pane.getChildren().addAll(circle, rectangle, line);
        // Set up the Scene and Stage
        Scene scene = new Scene(pane, 500, 500);
        primaryStage.setTitle("Graphics Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Explanation:

- **Circle**: We create a circle with a specified center `(100, 100)` and radius `50`. The color is set to red.
- **Rectangle**: The rectangle has a position `(200, 50)` with width `200` and height `100`. The color is blue.
- **Line:** The line starts at `(0, 0)` and ends at `(400, 400)` with a green color.

3.12.3. Animation Basics in JavaFX

JavaFX provides the `Animation` class and several sub-classes like `Timeline`, `TranslateTransition`, `RotateTransition`, and `FadeTransition` for creating animations.

Task 2: Simple Animation Using TranslateTransition

In this task, we will animate a circle moving horizontally using the TranslateTransition class.

```
import javafx.animation.TranslateTransition;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;
import javafx.util.Duration;
public class AnimationDemo extends Application {
   @Override
   public void start(Stage primaryStage) {
      // Create a Pane to hold the circle
      Pane pane = new Pane();
      // Create a Circle to animate
      Circle circle = new Circle(50, 50, 30);
      circle.setFill(Color.RED);
      // Add the circle to the Pane
      pane.getChildren().add(circle);
      // Create a TranslateTransition to animate the circle's movement
      TranslateTransition transition = new TranslateTransition();
      transition.setNode(circle); // Set the node to animate
      transition.setByX(400); // Move the circle 400 units along the X-axis
      transition.setCycleCount(TranslateTransition.INDEFINITE); // Repeat the
animation indefinitely
      transition.setAutoReverse(true); // Reverse the direction when the animation
completes
      transition.setDuration(Duration.seconds(2)); // Set the duration for one cycle of
the animation
      // Start the animation
```

```
        transition.play();
        // Set up the Scene and Stage
        Scene scene = new Scene(pane, 500, 500);
        primaryStage.setTitle("Simple Animation");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Explanation:

- **TranslateTransition:** This is used to move the circle along the X-axis by 400 pixels. It has a duration of 2 seconds and repeats indefinitely.
- Auto-reverse: When the animation completes, it will reverse the direction, causing the circle to move back and forth.

3.12.4. Using Timeline for More Complex Animations

The Timeline class provides a way to define a sequence of key frames, each representing a specific moment in an animation.

Task 3: Animate Using Timeline

This task will animate a circle's position, changing both its X and Y coordinates.

```
import javafx.animation.KeyFrame;
import javafx.animation.Timeline;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;
import javafx.util.Duration;
public class TimelineAnimationDemo extends Application {
```

```
@Override
public void start(Stage primaryStage) {
    // Create a Pane to hold the circle
    Pane pane = new Pane();
    // Create a Circle to animate
    Circle circle = new Circle(50, 50, 30);
    circle.setFill(Color.RED);
    // Add the circle to the Pane
    pane.getChildren().add(circle);
    // Create KeyFrames for the animation
    KeyFrame keyFrame1 = new KeyFrame(Duration.ZERO, e -> {
        circle.setCenterX(50);
        circle.setCenterY(50);
    });
    KeyFrame keyFrame2 = new KeyFrame(Duration.seconds(2), e -> {
        circle.setCenterX(400);
        circle.setCenterY(200);
    });
    KeyFrame keyFrame3 = new KeyFrame(Duration.seconds(4), e -> {
        circle.setCenterX(50);
        circle.setCenterY(50);
    });
    // Create a Timeline with the KeyFrames
    Timeline timeline = new Timeline(keyFrame1, keyFrame2, keyFrame3);
    timeline.setCycleCount(Timeline.INDEFINITE); // Repeat indefinitely
    timeline.setAutoReverse(true); // Reverse the animation after each cycle
    // Start the animation
    timeline.play();
    // Set up the Scene and Stage
    Scene scene = new Scene(pane, 500, 500);
    primaryStage.setTitle("Timeline Animation");
```

```
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Explanation:

- **KeyFrame:** Each KeyFrame defines the state of the circle at a particular time. The first key frame sets the position to `(50, 50)`, the second moves it to `(400, 200)`, and the third brings it back to `(50, 50)`.
- **Timeline**: A sequence of key frames that is played as an animation. The animation repeats indefinitely and reverses direction each time it finishes.

3.12.5. Handling User Input in Animations

You can interact with animations based on user input (mouse clicks, key presses, etc.).

Task 4: Animate on Mouse Click

This task animates a circle when the user clicks anywhere on the window.

```
import javafx.animation.TranslateTransition;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.input.MouseEvent;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;
import javafx.util.Duration;
public class MouseClickAnimationDemo extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create a Pane to hold the circle
        Pane pane = new Pane();
```

```
        // Create a Circle to animate
        Circle circle = new Circle(50, 50, 30);
        circle.setFill(Color.RED);
        // Add the circle to the Pane
        pane.getChildren().add(circle);
        // Create a TranslateTransition to animate the circle's movement
        TranslateTransition transition = new TranslateTransition();
        transition.setNode(circle);
        transition.setByX(200); // Move the circle 200 units along the X-axis
        transition.setByY(200); // Move the circle 200 units along the Y-axis
        transition.setDuration(Duration.seconds(1
    ));
      // Animate the circle when the mouse is clicked
        pane.setOnMouseClicked((MouseEvent event) -> {
            transition.play(); // Start the animation on click
        });
    // Set up the Scene and Stage
        Scene scene = new Scene(pane, 500, 500);
        primaryStage.setTitle("Mouse Click Animation");
        primaryStage.setScene(scene);
        primaryStage.show();
      }
      public static void main(String[] args) {
        launch(args);
      }
    }
```

Explanation:

- **MouseEvent**: The setOnMouseClicked method listens for mouse clicks on the pane. When the user clicks, the TranslateTransition animates the circle's movement.

Summery

In this lab, you learned how to create basic graphics and animations using JavaFX. You explored:

- Drawing shapes like circles, rectangles, and lines.
- Using transitions like TranslateTransition to move objects.
- Creating more complex animations using Timeline.
- Handling user interactions to trigger animations.

By using JavaFX's animation classes, you can create engaging and interactive graphical applications.

Additional Exercises

1. Animate a Rectangle: Create an animation where a rectangle grows in size or changes color over time.
2. Use RotateTransition: Animate the rotation of a shape around its center.
3. Interactive Animations: Combine mouse and keyboard inputs to trigger different animations.
4. Create a Game Animation: Build a simple animation for a moving object or character in a game-like scenario.

This concludes the Graphics and Animation lab for JavaFX. You can now explore more advanced animations and graphic effects to build interactive user interfaces!

Chapter Four

4. File Input-Output

Objective:

To understand how to perform file input/output (I/O) operations in Java using classes and methods from the java.io package. This will include reading from and writing to files, handling exceptions, and managing resources effectively.

4.1. Introduction to File I/O in Java

In Java, file I/O operations are handled using classes from the java.io package. The two main types of file operations are:

- File Input: Reading data from files.
- File Output: Writing data to files.

4.2. File Handling Classes in Java

File class: Represents a file or directory in the file system. It provides methods for creating, deleting, and checking the properties of files and directories.

FileReader: Used for reading character files.

FileWriter: Used for writing characters to files.

BufferedReader: Reads text from an input stream efficiently.

BufferedWriter: Writes text to an output stream efficiently.

PrintWriter: A subclass of Writer, which provides convenient methods for printing formatted representations of objects to a text-output stream.

FileInputStream: Used for reading byte data from a file.

FileOutputStream: Used for writing byte data to a file.

4.3. Basic Operations: File Creation, Reading, Writing, and Deleting

Creating a File

```
import java.io.File;
import java.io.IOException;
public class FileCreation {
    public static void main(String[] args) {
        try {
            File file = new File("example.txt");
```

```
            if (file.createNewFile()) {
                System.out.println("File created: " + file.getName());
            } else {
                System.out.println("File already exists.");
            }
        } catch (IOException e) {
            System.out.println("An error occurred.");
            e.printStackTrace();
        }
    }
}
```

Writing to a File

```
import java.io.FileWriter;
import java.io.IOException;
public class FileWriting {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("example.txt");
            writer.write("Hello, World!");
            writer.close();
            System.out.println("Data written to the file.");
        } catch (IOException e) {
            System.out.println("An error occurred.");
            e.printStackTrace();
        }
    }
}
```

Reading from a File

```
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;
```

```
public class FileReading {
    public static void main(String[] args) {
        try {
BufferedReader reader = new BufferedReader(new FileReader("example.txt"));
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
            reader.close();
        } catch (IOException e) {
            System.out.println("An error occurred.");
            e.printStackTrace();
        }
    }
}
```

Deleting a File

```
import java.io.File;
public class FileDeletion {
    public static void main(String[] args) {
        File file = new File("example.txt");
        if (file.delete()) {
            System.out.println("File deleted successfully.");
        } else {
            System.out.println("Failed to delete the file.");
        }
    }
}
```

4.4. Handling File Input/Output

4.4.1. Writing Multiple Lines using BufferedWriter

```
import java.io.BufferedWriter;
import java.io.FileWriter;
```

```
import java.io.IOException;
public class BufferedFileWriting {
    public static void main(String[] args) {
        try {
         BufferedWriter writer = new BufferedWriter(new FileWriter("example.txt"));
            writer.write("Hello, ");
            writer.newLine();
            writer.write("World!");
            writer.close();
            System.out.println("Data written to the file.");
        } catch (IOException e) {
            System.out.println("An error occurred.");
            e.printStackTrace();
        }
    }
}
```

4.5. Reading and Writing Byte Data using FileInputStream and FileOutputStream

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class ByteFileIO {
    public static void main(String[] args) {
        try {
            FileInputStream inputStream = new FileInputStream("source.txt");
            FileOutputStream outputStream = new FileOutputStream("destination.txt");
            int byteData;
            while ((byteData = inputStream.read()) != -1) {
                outputStream.write(byteData);
            }
```

```
            inputStream.close();
            outputStream.close();
            System.out.println("File copied successfully.");
        } catch (IOException e) {
            System.out.println("An error occurred.");
            e.printStackTrace();
        }
    }
}
```

Using PrintWriter for Formatted Output

```
import java.io.PrintWriter;
import java.io.IOException;
public class PrintWriterExample {
    public static void main(String[] args) {
        try {
            PrintWriter writer = new PrintWriter("example.txt");
            writer.println("This is an example of formatted output.");
            writer.printf("The value of PI is approximately %.2f", Math.PI);
            writer.close();
            System.out.println("Formatted data written to the file.");
        } catch (IOException e) {
            System.out.println("An error occurred.");
            e.printStackTrace();
        }
    }
}
```

4.6. Handling Exceptions in File I/O

When dealing with file I/O operations, errors may occur due to various reasons (e.g., file not found, read/write errors). Java handles exceptions using the try-catch block. Additionally, file

handling exceptions like FileNotFoundException and IOException should be properly caught and handled.

Example:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
public class FileNotFoundExample {
    public static void main(String[] args) {
        try {
            File file = new File("nonexistentfile.txt");
            FileReader reader = new FileReader(file);
            reader.read();
            reader.close();
        } catch (FileNotFoundException e) {
            System.out.println("File not found. Please check the file path.");
        } catch (IOException e) {
  System.out.println("An error occurred during file reading.");
        }
    }
}
```

Lab Exercises

Exercise 1: Write a program to create a text file and write multiple lines of text to it. Then, read the contents from the file and print them to the console.

Exercise2: Write a program to copy the contents of one file to another using `FileInputStream` and `FileOutputStream`.

Exercise3: Write a program to count the number of lines in a file using `BufferedReader`.

Exercise 4: Write a program that appends data to an existing file using `FileWriter` with append mode.

Summery

In this lab, you have learned the fundamentals of file I/O in Java, including the creation, reading, writing, and deletion of files. You have also practiced error handling and using various file I/O classes such as **BufferedReader**, **BufferedWriter**, **PrintWriter**, **FileReader,** and **FileWriter**.

By mastering file I/O, you are equipped to work with persistent data in Java applications, which is critical in many real-world scenarios such as logging, configuration management, and data storage.

Note: Always remember to close your streams (readers, writers, etc.) to prevent memory leaks. You can also use the **try-with-resources** statement (introduced in Java 7) to automatically close resources.

4.7. BufferedWriter and BufferedReader

BufferedWriter and BufferedReader are classes in Java used to improve the efficiency of input and output (I/O) operations by buffering the data. These classes are part of the java.io package and provide a more efficient way to write and read characters to and from a stream

Objective: To understand how to use the **BufferedWriter** and **BufferedReader** classes in Java for efficient reading and writing of text data. These classes are used to improve the performance of reading from and writing to files or streams.

4.7.1. Introduction to BufferedReader and BufferedWriter

BufferedReader

The BufferedReader class is used to read text from a file or input stream in an efficient manner. It buffers the input to minimize the number of read operations. This makes it faster when reading large files or multiple lines of text.

Common Methods:

- read(): Reads a single character.
- readLine(): Reads a whole line of text.
- close(): Closes the stream.

Example:

```
import java.io.*;
```

```
public class BufferedReaderExample {
    public static void main(String[] args) {
    try(BufferedReader reader = new BufferedReader(new FileReader("output.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        }
     catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

BufferedWriter

The BufferedWriter class is used to write text to a file or output stream in an efficient manner by buffering the characters. This reduces the number of I/O operations, as it writes a large block of characters at once instead of one character at a time.

Common Methods:

- write(int c): Writes a single character.
- write(char[] cbuf): Writes an array of characters.
- newLine(): Writes a line separator to the stream.
- close(): Closes the stream.

Example:

```
import java.io.*;
public class BufferedWriterExample {
    public static void main(String[] args) {
        try(BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"))){
            writer.write("Hello, World!");
            writer.newLine();
            writer.write("BufferedWriter example.");
        }
```

```
        catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

4.7.2. Basic Operations: Using BufferedReader and BufferedWriter

Creating a File and Writing to It Using BufferedWriter

Objective: Write a simple text to a file using BufferedWriter.

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
public class BufferedWriterExample {
    public static void main(String[] args) {
        try {
            BufferedWriter writer = new BufferedWriter(new FileWriter("example.txt"));
            writer.write("Hello, this is a simple text.");
            writer.newLine();  // Adds a new line
            writer.write("This is the second line.");
            writer.close();  // Always close the stream
            System.out.println("Data written to the file successfully.");
        }
         catch (IOException e) {
            System.out.println("An error occurred while writing to the file.");
            e.printStackTrace();
        }
    }
}
```

Reading from a File Using BufferedReader

Objective:

Read the content of the file created in the previous example using BufferedReader.

```
import java.io.BufferedReader;
```

```
import java.io.FileReader;
import java.io.IOException;
public class BufferedReaderExample {
    public static void main(String[] args) {
        try {
        BufferedReader reader = new BufferedReader(new FileReader("example.txt"));
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
            reader.close();  // Always close the stream
        }
         catch (IOException e) {
            System.out.println("An error occurred while reading the file.");
            e.printStackTrace();
        }
    }
}
```

Writing Multiple Lines Using BufferedWriter

Objective: Write multiple lines of text to a file using BufferedWriter.

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
public class MultiLineBufferedWriterExample {
    public static void main(String[] args) {
        try {
  BufferredWriter writer=new BufferedWriter(new FileWriter("multi_line_example.txt"));
            writer.write("This is the first line.");
            writer.newLine();
            writer.write("This is the second line.");
            writer.newLine();
```

```
            writer.write("This is the third line.");
            writer.close();  // Always close the stream
            System.out.println("Multiple lines written to the file successfully.");
        }
         catch (IOException e) {
            System.out.println("An error occurred while writing to the file.");
            e.printStackTrace();
        }
    }
}
```

Reading and Counting Lines in a File Using BufferedReader

Objective: Read a file and count how many lines it contains using BufferedReader.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
public class CountLinesBufferedReader {
    public static void main(String[] args) {
        try {
BufferedReader reader = new BufferedReader(new FileReader("multi_line_example.txt"));
            int lineCount = 0;
            while (reader.readLine() != null) {
                lineCount++;
            }
            reader.close();  // Always close the stream
            System.out.println("Total number of lines: " + lineCount);
        } catch (IOException e) {
            System.out.println("An error occurred while reading the file.");
            e.printStackTrace();
        }
    }
}
```

4.7.3. Advanced Operations: Combining BufferedReader and BufferedWriter

Copying Content from One File to Another

Objective: Use both BufferedReader and BufferedWriter to copy the content from one file to another efficiently.

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class FileCopyWithBufferedIO {
    public static void main(String[] args) {
        try {
         BufferedReader reader = new BufferedReader(new FileReader("source.txt"));
         BufferedWriter writer = new BufferedWriter(new FileWriter("destination.txt"));
            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line);
                writer.newLine();
            }
            reader.close();  // Close the reader
            writer.close();  // Close the writer
            System.out.println("File copied successfully.");
        } catch (IOException e) {
            System.out.println("An error occurred during file copy.");
            e.printStackTrace();
        }
    }
}
```

Reading User Input from Console Using BufferedReader

Objective: Read user input from the console using BufferedReader and then write it to a file using BufferedWriter.

```
import java.io.BufferedReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.InputStreamReader;
public class ConsoleInputToFile {
    public static void main(String[] args) {
        try{
BufferedReader consoleReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter fileWriter = new BufferedWriter(new FileWriter("user_input.txt"));
            System.out.println("Enter some text (type 'exit' to stop):");
            String userInput;
            while (!(userInput = consoleReader.readLine()).equals("exit")) {
                fileWriter.write(userInput);
                fileWriter.newLine();
            }
            consoleReader.close();
            fileWriter.close();
            System.out.println("User input written to the file.");
        } catch (IOException e) {
            System.out.println("An error occurred while reading user input or writing to
the file.");
            e.printStackTrace();
        }
    }
}
```

Lab Exercises

Exercise 1: File Writing and Reading

Write a program to:

1. Create a file named student_info.txt.
2. Use BufferedWriter to write the following information into the file:
   - Name: John Doe

- Age: 21
- Major: Computer Science

3. Use BufferedReader to read and display the content of the file.

Exercise 2: Counting Words in a File

Write a program that reads a file containing multiple lines and counts the number of words in it. Assume that words are separated by spaces.

Exercise 3: Reversing File Content

Write a program to:

1. Read the content of a file using BufferedReader.
2. Reverse the content of the file line by line.
3. Write the reversed content to a new file using `BufferedWriter`.

Summery

In this lab, you learned how to efficiently handle file reading and writing in Java using BufferedReader and BufferedWriter. These classes are essential for improving the performance of I/O operations by buffering input and output data. You also practiced using these classes to perform tasks like copying files, writing and reading multiple lines, and handling user input.

Buffered I/O is useful for handling larger files, as it reduces the number of I/O operations, leading to improved performance. It is an essential tool when dealing with text-based file manipulation in Java.

Note: Always ensure you close your BufferedReader and BufferedWriter objects to release the resources. You can also use the try-with-resources statement to handle this automatically.

4.8. DataInputStream and DataOutputStream

Objective:

To learn how to use DataInputStream and DataOutputStream classes in Java to read and write primitive data types (like int, double, float, etc.) in a portable and efficient way. These classes are part of the **java.io** package and provide methods to read and write primitive data types directly from and to a file or stream.

4.8.1. Introduction to DataInputStream and DataOutputStream

DataInputStream

The DataInputStream class allows you to read primitive data types (int, float, double, long, etc.) from an input stream in a machine-independent way.

Common Methods:

- readBoolean(): Reads a `boolean` value.
- readByte(): Reads a `byte`.
- readChar(): Reads a `char`.
- readDouble(): Reads a `double`.
- readInt(): Reads an `int`.
- readLong(): Reads a `long`.
- readFloat(): Reads a `float`.
- readUTF(): Reads a `String` in modified UTF-8 encoding.

DataOutputStream

The DataOutputStream class allows you to write primitive data types to an output stream in a machine-independent manner.

Common Methods:

- writeBoolean(boolean v): Writes a `boolean` value.
- writeByte(int v): Writes a `byte`.
- writeChar(int v): Writes a `char`.
- writeDouble(double v): Writes a `double`.
- writeInt(int v): Writes an `int`.
- writeLong(long v): Writes a `long`.
- writeFloat(float v): Writes a `float`.
- writeUTF(String str): Writes a `String` in modified UTF-8 encoding.

These classes are particularly useful when you need to read and write binary data that is platform-independent.

4.8.2. Basic Operations: Using DataInputStream and DataOutputStream

Writing Primitive Data Types Using DataOutputStream

Objective: Write various primitive data types to a file using DataOutputStream.

```
import java.io.DataOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class DataOutputStreamExample {
   public static void main(String[] args) {
            try{DataOutputStream outputStream = new DataOutputStream(new
FileOutputStream("datafile.dat"));
         // Writing different types of data
         outputStream.writeInt(123);       // Writes an integer
         outputStream.writeDouble(45.67);  // Writes a double
         outputStream.writeBoolean(true);  // Writes a boolean
         outputStream.writeUTF("Hello, World!");  // Writes a string
         outputStream.close();
         System.out.println("Data written to the file successfully.");
      } catch (IOException e) {
         System.out.println("An error occurred while writing to the file.");
         e.printStackTrace();
      }
   }
}
```

Reading Primitive Data Types Using DataInputStream

Objective: Read the primitive data types that were written to a file using DataInputStream.

```
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.IOException;
public class DataInputStreamExample {
   public static void main(String[] args) {
            try{
DataInputStream inputStream = new DataInputStream(new FileInputStream("datafile.dat"));
         // Reading data
         int intValue = inputStream.readInt();        // Reads an integer
```

```
            double doubleValue = inputStream.readDouble(); // Reads a double
            boolean booleanValue = inputStream.readBoolean(); // Reads a boolean
            String stringValue = inputStream.readUTF();    // Reads a string
            System.out.println("Integer: " + intValue);
            System.out.println("Double: " + doubleValue);
            System.out.println("Boolean: " + booleanValue);
            System.out.println("String: " + stringValue);
            inputStream.close();
        } catch (IOException e) {
            System.out.println("An error occurred while reading the file.");
            e.printStackTrace();
        }
    }
}
```

4.8.3. Advanced Operations: Working with Multiple Data Types

Writing and Reading Multiple Data Types

Objective: Write and read multiple data types in one program using DataOutputStream and DataInputStream.

```
import java.io.DataOutputStream;
import java.io.DataInputStream;
import java.io.FileOutputStream;
import java.io.FileInputStream;
import java.io.IOException;
public class DataStreamMultipleTypes {
    public static void main(String[] args) {
        // Writing multiple data types to a file
        Try{ (DataOutputStream outputStream = new DataOutputStream(new
FileOutputStream("multiData.dat"))) {
            outputStream.writeInt(100);          // Integer
            outputStream.writeDouble(45.56);      // Double
            outputStream.writeFloat(12.34f);      // Float
```

```
            outputStream.writeLong(123456789L);    // Long
            outputStream.writeUTF("Java I/O");    // String
            System.out.println("Data written successfully.");
        } catch (IOException e) {
            System.out.println("An error occurred while writing.");
            e.printStackTrace();
        }
        // Reading multiple data types from the file
        Try{ (DataInputStream inputStream = new DataInputStream(new
FileInputStream("multiData.dat"))) {
            int intValue = inputStream.readInt();
            double doubleValue = inputStream.readDouble();
            float floatValue = inputStream.readFloat();
            long longValue = inputStream.readLong();
            String stringValue = inputStream.readUTF();
            System.out.println("Read from file:");
            System.out.println("Integer: " + intValue);
            System.out.println("Double: " + doubleValue);
            System.out.println("Float: " + floatValue);
            System.out.println("Long: " + longValue);
            System.out.println("String: " + stringValue);
        } catch (IOException e) {
            System.out.println("An error occurred while reading.");
            e.printStackTrace();
        }
    }
}
```

Lab Exercises

Exercise 1: Write Primitive Data Types to a File

1. Write a Java program that uses DataOutputStream to write at least five different primitive data types (such as int, double, boolean, char, and String) to a file.

2. Read the data back from the file using DataInputStream and display the contents on the console.

Exercise 2: Store Student Information

1. Write a program that stores the following information about a student in a file:
   - Name (String)
   - Age (int)
   - GPA (double)

2. Use DataOutputStream to store the information in the file.

3. Write another program to read the data back from the file and display it.

Exercise 3: Data File Copying

1. Create a file containing mixed data types (e.g., int, double, String, etc.) using DataOutputStream.

2. Write a program to read the data from the original file using DataInputStream and write it to a new file using DataOutputStream.

Summery

In this lab, you learned how to use the DataInputStream and DataOutputStream classes to read and write primitive data types in a machine-independent format. These streams are particularly useful when you need to store binary data that must be portable across different platforms.

The DataInputStream and DataOutputStream classes handle the complexity of converting primitive data types to and from their raw binary representations, which can be used to efficiently store and retrieve data in a binary format.

4.9. Multi-threading Concepts

4.9.1. Extending Thread Class vs. Implementing Runnable Interface

Objective:

- Understand the difference between creating threads by extending the Thread class and implementing the Runnable interface.
- Compare the flexibility and use cases of both methods.

Extending the Thread Class

Steps:

1. Create a class that extends **Thread.**
2. Override the **run()** method to define the task.
3. Create thread objects and start them.

Code Example:

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " - Count: " + i);
            try {
                Thread.sleep(500); // Pause for 500ms
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted.");
            }
        }
    }
}
public class ThreadExample {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.setName("Thread-A");
        t2.setName("Thread-B");
        t1.start();
        t2.start();
    }
}
```

Implementing the Runnable Interface

Steps:

1. Create a class that implements **Runnable.**
2. Implement the **run()** method to define the task.
3. Pass the **Runnable** object to a **Thread** constructor and start it.

Code Example:

```
class MyRunnable implements Runnable {
        public void run() {
            for (int i = 1; i <= 5; i++) {
                System.out.println(Thread.currentThread().getName() + " - Count: " + i);
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    System.out.println("Thread interrupted.");
                }
            }
        }
    }
    public class RunnableExample {
        public static void main(String[] args) {
            MyRunnable task = new MyRunnable();
            Thread t1 = new Thread(task, "Thread-X");
            Thread t2 = new Thread(task, "Thread-Y");
            t1.start();
            t2.start();
        }
    }
```

Tasks:

- Compare the flexibility of the two methods.
- Discuss why implementing Runnable is preferred in cases of multiple inheritances.

4.10. Thread Synchronization

Objective:

- Learn how to avoid thread interference using the synchronized keyword.
- Ensure proper access to shared resources in multi-threaded programs.

Steps:

Problem without Synchronization:

1. Create a class with a shared resource.
2. Access the resource from multiple threads without synchronization.
3. Observe the inconsistent output.

Code Example:

```
class Counter {
    private int count = 0;
    public void increment() {
        count++;
        System.out.println(Thread.currentThread().getName() + " - Count: " + count);
    }
}
class CounterThread extends Thread {
    Counter counter;
    CounterThread(Counter counter) {
        this.counter = counter;
    }
    public void run() {
        for (int i = 0; i < 5; i++) {
            counter.increment();
        }
    }
}
public class WithoutSyncExample {
    public static void main(String[] args) {
        Counter counter = new Counter();
        CounterThread t1 = new CounterThread(counter);
        CounterThread t2 = new CounterThread(counter);
```

```
        t1.start();
        t2.start();
    }
}
```

Solution Using Synchronization: Use the synchronized keyword in the method or block to ensure only one thread can access the resource at a time.

Code Example:

```
class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
        System.out.println(Thread.currentThread().getName() + " - Count: " + count);
    }
}
class CounterThread extends Thread {
    Counter counter;
    CounterThread(Counter counter) {
        this.counter = counter;
    }
    public void run() {
        for (int i = 0; i < 5; i++) {
            counter.increment();
        }
    }
}
public class WithSyncExample {
    public static void main(String[] args) {
        Counter counter = new Counter();
        CounterThread t1 = new CounterThread(counter);
        CounterThread t2 = new CounterThread(counter);
        t1.start();
```

```
            t2.start();
        }
    }
```

Static Synchronization: Synchronize methods at the class level using static synchronized.

Tasks:

- Run both versions (with and without synchronization) and observe the difference.
- Discuss scenarios where synchronization is critical.

Thread Synchronization with wait() and notify()

Objective: Implement inter-thread communication using wait(), notify(), and notifyAll().

Steps: Producer-Consumer Problem:

1. Create a shared resource class.
2. Use wait() to pause threads and notify() to resume them.

Code Example:

```
class SharedResource {
 private int data;
 private boolean available = false;
 public synchronized void produce(int value) {
    while (available) {
       try {
          wait();
       } catch (InterruptedException e) {
          System.out.println("Interrupted");
       }
    }
    data = value;
    available = true;
    System.out.println("Produced: " + data);
    notify();
 }
 public synchronized void consume() {
    while (!available) {
```

```
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println("Interrupted");
            }
        }
        System.out.println("Consumed: " + data);
        available = false;
        notify();
    }
}
class Producer extends Thread {
    SharedResource resource;
    Producer(SharedResource resource) {
        this.resource = resource;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            resource.produce(i);
        }
    }
}
class Consumer extends Thread {
    SharedResource resource;
    Consumer(SharedResource resource) {
        this.resource = resource;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            resource.consume();
        }
```

```
    }
}
public class InterThreadCommunicationExample {
    public static void main(String[] args) {
        SharedResource resource = new SharedResource();
        Producer producer = new Producer(resource);
        Consumer consumer = new Consumer(resource);
        producer.start();
        consumer.start();
    }
}
```

Tasks:

- Modify the program to add multiple producers and consumers.
- Discuss deadlock scenarios and how they can be avoided.

Summery

This lab manual covers the foundational concepts of multi-threading in Java, focusing on creating threads, synchronization, and inter-thread communication. These experiments will help you build robust multi-threaded applications.

Chapter Five

5. Socket Programming

5.1. Introduction to Socket and ServerSocket Classes

Objective:

- Understand the components of the Socket and ServerSocket classes.
- Implement client-server communication using sockets.

Components of Socket Class: The Socket class in Java provides methods to connect to a server and send/receive data. Key components include:

- **Socket(String host, int port):** Creates a socket to connect to the specified host and port.
- **getInputStream():** Returns the input stream to receive data.
- **getOutputStream():** Returns the output stream to send data.
- **close():** Closes the socket connection.

Components of ServerSocket Class: The ServerSocket class listens for incoming client connections. Key components include:

- **ServerSocket(int port):** Creates a server socket bound to the specified port.
- **accept():** Waits for a client to connect and returns a Socket object for communication.
- **close():** Closes the server socket.

5.1.1. Implementing a Simple Client-Server Program

Objective:

- Create a server program that listens for client connections.
- Create a client program to connect to the server and exchange messages.

Steps:

1. Start the server program.
2. Run the client program to connect to the server.
3. Exchange messages between client and server.

Server Program:

```
import java.io.*;
import java.net.*;
```

```
public class SimpleServer {
    public static void main(String[] args) {
        try {
            // Create a server socket on port 5000
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("Server is running and waiting for a client...");
            // Accept a client connection
            Socket socket = serverSocket.accept();
            System.out.println("Client connected.");
            // Get input and output streams
            InputStream input = socket.getInputStream();
           BufferedReader reader = new BufferedReader(new InputStreamReader(input));
            OutputStream output = socket.getOutputStream();
            PrintWriter writer = new PrintWriter(output, true);
            // Read message from client and respond
            String message = reader.readLine();
            System.out.println("Received from client: " + message);
            writer.println("Hello from server!");
            // Close streams and sockets
            reader.close();
            writer.close();
            socket.close();
            serverSocket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Client Program:

```
import java.io.*;
import java.net.*;
```

```
public class SimpleClient {
    public static void main(String[] args) {
        try {
            // Connect to the server at localhost on port 5000
            Socket socket = new Socket("localhost", 5000);
            // Get input and output streams
            InputStream input = socket.getInputStream();
           BufferedReader reader = new BufferedReader(new InputStreamReader(input));
            OutputStream output = socket.getOutputStream();
            PrintWriter writer = new PrintWriter(output, true);
            // Send message to server
            writer.println("Hello from client!");
            // Read response from server
            String response = reader.readLine();
            System.out.println("Received from server: " + response);
            // Close streams and socket
            reader.close();
            writer.close();
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Tasks:

- Modify the program to send multiple messages back and forth.
- Observe the sequence of messages and understand the client-server interaction.

5.1.2. Multi-Client Handling with ServerSocket

Objective: Implement a server that can handle multiple clients simultaneously using threads.

Steps:

1. Create a server program that uses threads to handle multiple clients.

2. Each client communicates with the server independently.

Multi-Client Server Program:

```
import java.io.*;
import java.net.*;
class ClientHandler extends Thread {
  private Socket socket;
   ClientHandler(Socket socket) {
      this.socket = socket;
   }
   public void run() {
      try {
         // Get input and output streams
         InputStream input = socket.getInputStream();
        BufferedReader reader = new BufferedReader(new InputStreamReader(input));
         OutputStream output = socket.getOutputStream();
         PrintWriter writer = new PrintWriter(output, true);
         // Communicate with the client
         String message;
         while ((message = reader.readLine()) != null) {
            System.out.println("Received from client: " + message);
            writer.println("Server received: " + message);
         }
         // Close streams and socket
         reader.close();
         writer.close();
         socket.close();
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

```
public class MultiClientServer {
    public static void main(String[] args) {
        try {
            // Create a server socket on port 6000
            ServerSocket serverSocket = new ServerSocket(6000);
            System.out.println("Server is running and waiting for clients...");
            while (true) {
                // Accept a client connection
                Socket socket = serverSocket.accept();
                System.out.println("Client connected.");
                // Handle the client in a separate thread
                new ClientHandler(socket).start();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Client Program (Same as Above): Use the SimpleClient program, but connect multiple clients to the server simultaneously.

Tasks:

- Run multiple instances of the client program and observe the server's behavior.
- Modify the server to broadcast a message to all connected clients.

5.1.3. File Transfer Using Sockets

Objective: Implement a server-client application for file transfer using sockets.

Steps:

1. Create a server program to send a file.
2. Create a client program to receive and save the file.

File Server Program:

```
import java.io.*;
import java.net.*;
```

```
public class FileServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(7000);
            System.out.println("File server is running...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected.");
            // Send file
            File file = new File("sample.txt");
            FileInputStream fileInput = new FileInputStream(file);
            OutputStream output = socket.getOutputStream();
            byte[] buffer = new byte[4096];
            int bytesRead;
            while ((bytesRead = fileInput.read(buffer)) != -1) {
                output.write(buffer, 0, bytesRead);
            }
            System.out.println("File sent successfully.");
            fileInput.close();
            output.close();
            socket.close();
            serverSocket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

File Client Program:

```
import java.io.*;
import java.net.*;
public class FileClient {
    public static void main(String[] args) {
```

```
        try {
            Socket socket = new Socket("localhost", 7000);
            InputStream input = socket.getInputStream();
            // Save file
            FileOutputStream fileOutput = new FileOutputStream("received_sample.txt");
            byte[] buffer = new byte[4096];
            int bytesRead;
            while ((bytesRead = input.read(buffer)) != -1) {
                fileOutput.write(buffer, 0, bytesRead);
            }
            System.out.println("File received successfully.");
            fileOutput.close();
            input.close();
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Tasks:

- Transfer different types of files (text, image, etc.).
- Observe file integrity after transfer.

Summery

This lab manual provides hands-on experience with socket programming, enabling you to build client-server applications in Advanced Java effectively. These experiments cover essential concepts like single-client, multi-client handling, and file transfer over sockets.

Chapter six

6. Remote Method Invocation (RMI)

6.1. Introduction to RMI

What is RMI?

Remote Method Invocation (RMI) allows Java programs to invoke methods on an object running in another JVM, possibly on a different physical machine. RMI abstracts the details of network communication, making distributed computing easier.

6.2. Architecture of RMI

The RMI architecture consists of three main layers:

A. Stub and Skeleton Layer:

- **Stub**: Client-side proxy that forwards method calls to the server.
- **Skeleton**: Server-side proxy that dispatches client calls to the actual remote object.

B. Remote Reference Layer: Handles references between client and remote objects.

C. Transport Layer: Manages network connections and the communication between JVMs.

6.3. Steps to Implement RMI

To implement RMI, follow these steps:

1. Define the remote interface.
2. Implement the remote interface.
3. Create the client program.
4. Compile and generate stubs using rmic.
5. Start the RMI Registry.
6. Run the server and client programs.

Simple RMI Implementation

Objective: Implement a simple RMI-based client-server application where the client invokes a method on the server to perform an addition operation.

Step-by-Step Implementation

1. Define the Remote Interface: This interface must extend java.rmi.Remote and declare remote methods with throws RemoteException.

```
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

2. Implement the Remote Interface: The implementation must extend UnicastRemoteObject.

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
public class CalculatorImpl extends UnicastRemoteObject implements
Calculator {
    public CalculatorImpl() throws RemoteException {
        super();
    }
    @Override
    public int add(int a, int b) throws RemoteException {
        return a + b;
    }
}
```

3. Create the Server Program: The server registers the remote object with the RMI registry.

```
import java.rmi.Naming;
import java.rmi.RemoteException;
public class CalculatorServer {
    public static void main(String[] args) {
        try {
            CalculatorImpl calc = new CalculatorImpl();
            Naming.rebind("rmi://localhost:5000/CalculatorService", calc);
            System.out.println("Server is ready.");
        } catch (Exception e) {
            e.printStackTrace();
```

```
        }
      }
    }
```

4. Create the Client Program: The client looks up the remote object in the RMI registry and invokes the method.

```
import java.rmi.Naming;
public class CalculatorClient {
  public static void main(String[] args) {
    try {
Calculator calc = (Calculator) Naming.lookup("rmi://localhost:5000/CalculatorService");
        int result = calc.add(10, 20);
        System.out.println("Result: " + result);
    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

5. Compile and Generate Stubs: Use the following commands:

1. Compile all classes:

```
javac *.java
```

2. Generate stub for CalculatorImpl:

```
rmic CalculatorImpl
```

5. Start the RMI Registry:

Start the RMI registry on port 5000:

```
rmiregistry 5000
```

7. Run the Server and Client:

1. Start the server:

```
java CalculatorServer
```

2. Run the client:

```
java CalculatorClient
```

Tasks:

- Modify the program to perform additional operations (subtraction, multiplication, division).
- Test the behavior when the server is unavailable.

Multi-Client RMI

Objective: Implement an RMI server that can handle requests from multiple clients simultaneously.

Steps:

1. Use the CalculatorImpl class from Experiment 1.
2. Create multiple client programs that invoke methods concurrently.

Multi-Client Client Program:

```
import java.rmi.Naming;
public class MultiClient {
    public static void main(String[] args) {
        try {
Calculator calc = (Calculator) Naming.lookup("rmi://localhost:5000/CalculatorService");
            // Client 1
            new Thread(() -> {
                try {
                    System.out.println("Client 1: " + calc.add(5, 7));
                } catch (Exception e) {
                 e.printStackTrace();
        }
            }).start();
            // Client 2
            new Thread(() -> {
                try {
                    System.out.println("Client 2: " + calc.add(15, 25));
                } catch (Exception e) {
                    e.printStackTrace();
                }
```

```
            }).start();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

}
```

Tasks:

- Observe how the server handles multiple clients.
- Modify the program to log client requests on the server.

RMI with File Transfer

Objective: Implement RMI to transfer a file from the server to the client.

Remote Interface:

```
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface FileServer extends Remote {
    byte[] downloadFile(String fileName) throws RemoteException;
}
```

Server Implementation:

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.nio.file.Files;
import java.nio.file.Paths;
public class FileServerImpl extends UnicastRemoteObject implements FileServer {
    public FileServerImpl() throws RemoteException {
        super();
    }
    @Override
    public byte[] downloadFile(String fileName) throws RemoteException {
        try {
            return Files.readAllBytes(Paths.get(fileName));
```

```
            } catch (Exception e) {
                e.printStackTrace();
                return null;
            }
        }
    }
```

Server Program:

```
    import java.rmi.Naming;
    public class FileServerApp {
        public static void main(String[] args) {
            try {
                FileServerImpl fileServer = new FileServerImpl();
                Naming.rebind("rmi://localhost:5000/FileService", fileServer);
                System.out.println("File server is ready.");
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
```

Client Program:

```
    import java.io.*;
    import java.rmi.Naming;
    public class FileClient {
        public static void main(String[] args) {
            try {
  FileServer fileServer = (FileServer) Naming.lookup("rmi://localhost:5000/FileService");
                byte[] data = fileServer.downloadFile("test.txt");
                if (data != null) {
                    FileOutputStream fos = new FileOutputStream("downloaded_test.txt");
                    fos.write(data);
                    fos.close();
```

```
                System.out.println("File downloaded successfully.");
            } else {
                System.out.println("File not found on server.");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Tasks:

- Test with different file types (text, images, etc.).
- Implement error handling for missing files.

Summery

This lab manual covers the foundational concepts of RMI, including architecture, single-client/multi-client implementation, and file transfer. These experiments demonstrate the power of RMI for distributed computing in Java.

Chapter Seven

7. Java Database Connectivity (JDBC)

This manual explains how to connect Java applications to a database using JDBC, execute SQL statements, and retrieve results.

7.1. Introduction

What is JDBC?

JDBC (Java Database Connectivity) is a Java API that allows Java applications to interact with databases. It provides methods to:

- Connect to a database.
- Execute SQL queries.
- Retrieve and manipulate results.

7.2. Setting up the Database

Objective: Set up a sample database for the lab.

Database Setup:

1. Install a Database:
   - Use MySQL, PostgreSQL, or any RDBMS of your choice.
   - For MySQL, download and install it from the [official MySQL website](https://www.mysql.com).
2. Create a Database: (Note: use database name **StudentDB** and table name **Student**).

```
CREATE DATABASE StudentDB;
```

3. Create a Table:

```
USE StudentDB;
CREATE TABLE Student (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   grade VARCHAR(5)
);
```

4. Insert Sample Data:

```
INSERT INTO Student (name, age, grade) VALUES
('Alice', 20, 'A'),
('Bob', 22, 'B'),
('Charlie', 21, 'A');
```

7.3. Connecting Java to a Database

Objective: Establish a connection between a Java application and a database using JDBC.

Steps:

1. Load the JDBC Driver:

Add the JDBC driver to your project. For MySQL, download the driver .jar file from the [MySQL Connector](https://dev.mysql.com/downloads/connector/j/) and add it to your project classpath.

2. Write Java Code to Connect:

```
import java.sql.Connection;
import java.sql.DriverManager;
public class DatabaseConnection {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/StudentDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password
        try {
          Connection connection = DriverManager.getConnection(url, user, password);
            System.out.println("Database connected successfully!");
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

7.4. Creating and Executing SQL Statements

Objective: Use JDBC to create and execute SQL queries.

Steps:

1. Read Data from the Database:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class ReadData {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/StudentDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password\
        try {
          Connection connection = DriverManager.getConnection(url, user, password);
            Statement statement = connection.createStatement();
            String query = "SELECT * FROM students";
            ResultSet resultSet = statement.executeQuery(query);
            while (resultSet.next()) {
                System.out.println("ID: " + resultSet.getInt("id"));
                System.out.println("Name: " + resultSet.getString("name"));
                System.out.println("Age: " + resultSet.getInt("age"));
                System.out.println("Grade: " + resultSet.getString("grade"));
                System.out.println("------------------");
            }
            resultSet.close();
            statement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

2. Insert Data into the Database:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
public class InsertData {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/StudentDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password
        try {
          Connection connection = DriverManager.getConnection(url, user, password);
            Statement statement = connection.createStatement();
String query = "INSERT INTO students (name, age, grade) VALUES ('Daisy', 23, 'B')";
            int rowsAffected = statement.executeUpdate(query);
            System.out.println("Rows inserted: " + rowsAffected);
            statement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

3. Update Data in the Database:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
  public class UpdateData {
   public static void main(String[] args) {
    String url = "jdbc:mysql://localhost:3306/StudentDB";
     String user = "root"; // Replace with your username
```

```
        String password = ""; // Replace with your password
        try {
            Connection connection = DriverManager.getConnection(url, user, password);
            Statement statement = connection.createStatement();
            String query = "UPDATE students SET grade = 'A+' WHERE name = 'Daisy'";
            int rowsAffected = statement.executeUpdate(query);
            System.out.println("Rows updated: " + rowsAffected);
            statement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

4. Delete Data from the Database:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
public class DeleteData {
  public static void main(String[] args) {
      String url = "jdbc:mysql://localhost:3306/StudentDB";
      String user = "root"; // Replace with your username
      String password = ""; // Replace with your password
      try {
          Connection connection = DriverManager.getConnection(url, user, password);
          Statement statement = connection.createStatement();
          String query = "DELETE FROM students WHERE name = 'Daisy'";
          int rowsAffected = statement.executeUpdate(query);
          System.out.println("Rows deleted: " + rowsAffected);
          statement.close();
```

```
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Using Prepared Statements

Objective: Use PreparedStatement to execute parameterized queries.

Sample Code

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
public class PreparedStatementsExample {
    public static void main(String[] args) {
    String url = "jdbc:mysql://localhost:3306/StudentDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password
        try {
            Connection connection = DriverManager.getConnection(url, user, password);
           String query = "INSERT INTO students (name, age, grade) VALUES (?, ?, ?)";
            PreparedStatement preparedStatement = connection.prepareStatement(query);
            // Set parameters
            preparedStatement.setString(1, "Edward");
            preparedStatement.setInt(2, 24);
            preparedStatement.setString(3, "B");
            int rowsAffected = preparedStatement.executeUpdate();
            System.out.println("Rows inserted: " + rowsAffected);
            preparedStatement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
```

```
            }
        }
    }
```

Retrieving Metadata

Objective: Retrieve metadata about the database and its tables.

Sample Code:

```
import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
public class DatabaseMetadataExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/StudentDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password
        try {
            Connection connection = DriverManager.getConnection(url, user, password);
            DatabaseMetaData metaData = connection.getMetaData();
System.out.println("Database Product Name: " + metaData.getDatabaseProductName());
System.out.println("Database Product Version: " + metaData.getDatabaseProductVersion());
            System.out.println("Driver Name: " + metaData.getDriverName());
            System.out.println("Driver Version: " + metaData.getDriverVersion());
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Summery

This lab manual provides a step-by-step guide to setting up a database, connecting Java applications to it, and performing CRUD (Create, Read, Update, Delete) operations.

7.5. Querying and Manipulating Results of SQL Statements

This lab explores more advanced aspects of querying databases and manipulating the results using Java's JDBC API.

7.5.1. Querying Database and Processing Result

Objective:

- Write advanced SQL queries in Java.
- Process and display the results using JDBC.

Steps:

1. Create a Sample Database and Table:

Use the following SQL commands to set up the database:

Use database name **LibraryDB** and table name **books**.

```
CREATE DATABASE LibraryDB
USE LibraryDB;
CREATE TABLE books (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100),
    author VARCHAR(50),
    price DECIMAL(10, 2),
    year_published INT
        );
INSERT INTO books (title, author, price, year_published) VALUES
('Java Programming', 'John Doe', 500.00, 2015),
('Database Systems', 'Jane Smith', 600.50, 2018),
('Advanced Algorithms', 'Alice Brown', 700.75, 2012),
('Web Development', 'Robert White', 450.00, 2020);
```

2. Write Java Code to Query Data:

Use a SELECT query to retrieve and display book details.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
```

```
import java.sql.Statement;
public class QueryBooks {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/LibraryDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password
        try {
            Connection connection = DriverManager.getConnection(url, user, password);
            Statement statement = connection.createStatement();
            String query = "SELECT * FROM books";
            ResultSet resultSet = statement.executeQuery(query);
            System.out.println("Book Details:");
            while (resultSet.next()) {
                System.out.println("ID: " + resultSet.getInt("id"));
                System.out.println("Title: " + resultSet.getString("title"));
                System.out.println("Author: " + resultSet.getString("author"));
                System.out.println("Price: " + resultSet.getDouble("price"));
                System.out.println("Year Published: " + resultSet.getInt("year_published"));
                System.out.println("-------------------");
            }
            resultSet.close();
            statement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

7.5.2. Parameterized Queries Using PreparedStatement

Objective: Use PreparedStatement to execute parameterized queries for better security and efficiency.

Steps:

1. Write a Parameterized Query:

Query books published after a given year.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
public class QueryBooksByYear {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/LibraryDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password
        try {
          Connection connection = DriverManager.getConnection(url, user, password);
            String query = "SELECT * FROM books WHERE year_published > ?";
          PreparedStatement preparedStatement = connection.prepareStatement(query);
            preparedStatement.setInt(1, 2015);
            ResultSet resultSet = preparedStatement.executeQuery();
            System.out.println("Books Published After 2015:");
            while (resultSet.next()) {
                System.out.println("Title: " + resultSet.getString("title"));
                System.out.println("Author: " + resultSet.getString("author"));
      System.out.println("Year Published: " +       resultSet.getInt("year_published"));
                System.out.println("-------------------");
            }
            resultSet.close();
            preparedStatement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
```

```
    }
}
```

7.5.3. Manipulating Query Results

Objective: Modify data in the database using ResultSet objects.

Steps:

1. Enable ResultSet Updates:

Open the connection with updateable ResultSet support.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class UpdateBookPrice {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/LibraryDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password
        try {
          Connection connection = DriverManager.getConnection(url, user, password);
            Statement statement = connection.createStatement(
                    ResultSet.TYPE_SCROLL_INSENSITIVE,
                    ResultSet.CONCUR_UPDATABLE
            );
            String query = "SELECT * FROM books";
            ResultSet resultSet = statement.executeQuery(query);
            while (resultSet.next()) {
                double price = resultSet.getDouble("price");
                if (price < 500.00) {
                    resultSet.updateDouble("price", price + 50.00);
                    resultSet.updateRow();
                    System.out.println("Updated price for: " + resultSet.getString("title"));
                }
```

```
            }
            resultSet.close();
            statement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

7.5.4. Inserting New Rows Using ResultSet

Objective: Insert new rows into the database using `ResultSet`.

Steps:

1. Write Code for Inserting Rows:

Use ResultSet.moveToInsertRow() and ResultSet.insertRow().

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class InsertNewBook {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/LibraryDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password
        try {
          Connection connection = DriverManager.getConnection(url, user, password);
            Statement statement = connection.createStatement(
                    ResultSet.TYPE_SCROLL_INSENSITIVE,
                    ResultSet.CONCUR_UPDATABLE
            );
            String query = "SELECT * FROM books";
```

```
            ResultSet resultSet = statement.executeQuery(query);
            resultSet.moveToInsertRow();
            resultSet.updateString("title", "Spring Framework");
            resultSet.updateString("author", "Mark Fisher");
            resultSet.updateDouble("price", 750.00);
            resultSet.updateInt("year_published", 2019);
            resultSet.insertRow();
            System.out.println("New book added successfully!");
            resultSet.close();
            statement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

7.5.5. Deleting Rows Using ResultSet

Objective: Delete specific rows from the database using `ResultSet`.

Steps:

1. Write Code to Delete Rows:

Use ResultSet.deleteRow() to remove rows that meet a condition.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class DeleteOldBooks {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/LibraryDB";
        String user = "root"; // Replace with your username
        String password = ""; // Replace with your password
```

```
        try {
            Connection connection = DriverManager.getConnection(url, user, password);
            Statement statement = connection.createStatement(
                    ResultSet.TYPE_SCROLL_INSENSITIVE,
                    ResultSet.CONCUR_UPDATABLE
            );
            String query = "SELECT * FROM books WHERE year_published < 2015";
            ResultSet resultSet = statement.executeQuery(query);
            while (resultSet.next()) {
                System.out.println("Deleting book: " + resultSet.getString("title"));
                resultSet.deleteRow();
            }
            resultSet.close();
            statement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Summary

These experiments provide a deeper understanding of querying and manipulating database results using Java.

Chapter Eight

8. Servlet Programming

This lab explores servlets, their lifecycle, and how to compile, deploy, and access data from HTML forms. It also introduces HTTP request-response handling and HTTP codes.

8.1. Servlet Lifecycle

Objective: Understand the lifecycle of a servlet: init(), service(), and destroy().

Steps:

1. Create a Servlet Class:

Create a servlet that prints messages in each lifecycle method.

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/LifecycleServlet")
public class LifecycleServlet extends HttpServlet {
    @Override
    public void init() throws ServletException {
        System.out.println("Servlet is being initialized");
    }
    @Override
  protected void service(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.getWriter().println("Servlet is servicing the request.");
        System.out.println("Servlet is servicing a request.");
    }
    @Override
```

```
    public void destroy() {
        System.out.println("Servlet is being destroyed");
    }
}
```

2. Deploy and Test:
   - ❖ Compile the servlet and place it in the `webapps/ROOT/WEB-INF/classes` folder of your server (e.g., Apache Tomcat).
   - ❖ Access the servlet using a browser or a REST client:

http://localhost:8080/LifecycleServlet

8.2. Compiling and Deploying a Servlet

Objective: Learn how to compile and deploy a servlet in a web application.

Steps:

1. Setup Project Structure:

```
MyWebApp/
├── WEB-INF/
    ├── web.xml
    ├── classes/
    ├── lib/
```

2. Write a Servlet:

   Save the servlet (e.g.,HelloWorldServlet.java) in the WEB-INF/classes folder.

3. Create a Deployment Descriptor (`web.xml`):

```
<web-app>
    <servlet>
        <servlet-name>HelloWorldServlet</servlet-name>
        <servlet-class>HelloWorldServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>HelloWorldServlet</servlet-name>
```

```
        <url-pattern>/hello</url-pattern>
    </servlet-mapping>
</web-app>
```

4. Compile and Deploy:
   - ❖ Compile the servlet using javac.
   - ❖ Deploy the WAR file or directory to the `webapps` folder in Tomcat.
5. Access the Servlet:

http://localhost:8080/MyWebApp/hello

8.3. Accessing Data from HTML Form

Objective: Submit form data to a servlet and process it.

Steps:

1. Create an HTML Form:

```
<html>
<body>
    <form action="FormServlet" method="POST">
        Name: <input type="text" name="name"><br>
        Age: <input type="number" name="age"><br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

2. Create a Servlet to Process Form Data:

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/FormServlet")
public class FormServlet extends HttpServlet {
```

```
        @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String name = request.getParameter("name");
        String age = request.getParameter("age");
        response.setContentType("text/html");
        response.getWriter().println("<h1>Form Data Received</h1>");
        response.getWriter().println("<p>Name: " + name + "</p>");
        response.getWriter().println("<p>Age: " + age + "</p>");
    }
}
```

3. Test the Application:
   - Deploy the application.
   - Access the HTML form, submit data, and view the response.

8.4. Client Request and Server Response

This demonstration highlights the process of handling client requests and sending server responses in a Java Servlet. By understanding the **HttpServletRequest** and **HttpServletResponse** objects, you will learn how data is exchanged between the client and server.

Objective:

- Extract data from client requests.
- Send appropriate responses to the client.
- Understand HTTP headers, query parameters, and methods like GET and POST.

Steps:

1. **Create an HTML Form for Testing**

Save the following HTML file as request_form.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Client Request and Server Response</title>
```

```
</head>
<body>
    <h1>Client Request Form</h1>
    <form action="RequestResponseServlet" method="GET">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required><br><br>
        <label for="age">Age:</label>
        <input type="number" id="age" name="age" required><br><br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

2. **Create the Servlet**

RequestResponseServlet.java

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/RequestResponseServlet")
public class RequestResponseServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // Extracting Client Request Data
        String name = request.getParameter("name");
        String age = request.getParameter("age");
// Extracting Additional Request Details
        String clientIP = request.getRemoteAddr();
        String requestMethod = request.getMethod();
```

```
        String userAgent = request.getHeader("User-Agent");
        // Setting up Response
        response.setContentType("text/html");
        response.setCharacterEncoding("UTF-8");
        // Writing Response
        response.getWriter().println("<html><body>");
        response.getWriter().println("<h1>Server Response</h1>");
response.getWriter().println("<p>Hello, " + name + "! You are " + age + " years old.</p>");
        response.getWriter().println("<h2>Request Details</h2>");
        response.getWriter().println("<p>Client IP: " + clientIP + "</p>");
        response.getWriter().println("<p>Request Method: " + requestMethod +
    "</p>");
        response.getWriter().println("<p>User-Agent: " + userAgent + "</p>");
        response.getWriter().println("</body></html>");
      }
    }
```

3. Deploy the Servlet

a. Save the servlet file in your project's WEB-INF/classes directory.

b. Compile it using java`. For example:

```
javac -cp <path-to-servlet-api-jar> RequestResponseServlet.java
```

c. Update the `web.xml` file in the `WEB-INF` directory:

```
<web-app>
  <servlet>
    <servlet-name>RequestResponseServlet</servlet-name>
    <servlet-class>RequestResponseServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>RequestResponseServlet</servlet-name>
    <url-pattern>/RequestResponseServlet</url-pattern>
  </servlet-mapping>
</web-app>
```

4. Deploy the project in your servlet container (e.g., Apache Tomcat).
Test the Servlet

a. Open request_form.html in a browser.

b. Fill out the form and submit it.

c. The servlet will process the request and return a response.

Server Response

```
<html>
  <body>
    <h1>Server Response</h1>
    <p>Hello, John! You are 25 years old.</p>
    <h2>Request Details</h2>
    <p>Client IP: 127.0.0.1</p>
    <p>Request Method: GET</p>
    <p>User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36</p>
  </body>
</html>
```

Experimentation

1. Modify the Method

Change the form's method attribute to POST and implement the doPost method in the servlet.

```
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
  // Similar to doGet() logic
}
```

2. Return Different HTTP Codes

Return specific HTTP codes based on request parameters:

```
if (name == null || name.isEmpty()) {
```

```
        response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Name is
    missing");
    }
```

Key Concepts Demonstrated

1. **Client Request**:
   - ❖ Extract data using `request.getParameter()`.
   - ❖ Access headers, IP, and HTTP method.

2. Server Response:
   - ❖ Customize responses using `response.getWriter()`.
   - ❖ Set HTTP headers and content type.

3. HTTP Codes: Send appropriate status codes using response.setStatus() or response.sendError().

8.5. Understanding HTTP Codes

Objective: Demonstrate different HTTP status codes in a servlet.

Steps:

1. Create a Servlet to Respond with HTTP Codes:

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/HttpCodesServlet")
public class HttpCodesServlet extends HttpServlet {
    @Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    String code = request.getParameter("code");
        if (code == null) {
```

```
response.sendError(HttpServletResponse.SC_BAD_REQUEST, "No code provided");
                return;
            }
            switch (code) {
                case "200":
                    response.setStatus(HttpServletResponse.SC_OK);
                    response.getWriter().println("HTTP 200 OK");
                    break;
                case "404":
response.sendError(HttpServletResponse.SC_NOT_FOUND, "Resource not found");
                    break;
                case "500":
        response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERRO
        R, "Server error occurred");
                    break;
                default:
response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Unknown code");
            }
        }
    }
```

2. Test Different HTTP Codes:
   - Access the servlet with a query parameter,

Example:

http://localhost:8080/HttpCodesServlet?code=200

http://localhost:8080/HttpCodesServlet?code=404

Summary

These experiments provide hands-on experience with servlets, covering lifecycle methods, compiling and deploying servlets, handling form data, and working with HTTP request-response mechanics.

References:

1. S. Horstmann and Gary Cornell, Core Java 2 – Volume II- Advanced Features, Sun Microsystems Press
2. Harvey M. Deitel and Paul J. Deitel, Java How to Program, Deitel & Associates Inc.java.sun.com/docs/books/tutorial
3. Herbert Schildt 0-07-213084-9, complete reference java 2
4. Eckel, Bruce. Thinking in Java. 4th Ed. New Jersey: Prentice Hall